

МИНОБРНАУКИ РОССИИ
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ БЮДЖЕТНОЕ ОБРАЗОВАТЕЛЬНОЕ
УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
«БЕЛГОРОДСКИЙ ГОСУДАРСТВЕННЫЙ ТЕХНОЛОГИЧЕСКИЙ
УНИВЕРСИТЕТ им. В.Г. ШУХОВА»
(БГТУ им. В.Г. Шухова)

Институт информационных технологий и управляющих систем

Кафедра программного обеспечения вычислительной техники и автоматизированных систем

Направление подготовки 09.03.04 Программная инженерия

Направленность (профиль) образовательной программы Разработка программно-информационных систем

ВЫПУСКНАЯ КВАЛИФИКАЦИОННАЯ РАБОТА

на тему:

**Разработка сервиса хранения аудиоданных с применением алгоритмов
упаковки и потоковой распаковки**

Студент: Пахомов Владислав Андреевич
Зав. кафедрой: канд. техн. наук, доц. Поляков Владимир Михайлович
Руководитель: канд. физ.-мат. наук, доц. Осипов Олег Васильевич

К защите допустить
Зав. кафедрой _____ /Поляков В.М./

« _____ » _____ 2026 г.

Белгород 2026 г.

МИНОБРНАУКИ РОССИИ
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ БЮДЖЕТНОЕ ОБРАЗОВАТЕЛЬНОЕ
УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
«БЕЛГОРОДСКИЙ ГОСУДАРСТВЕННЫЙ ТЕХНОЛОГИЧЕСКИЙ
УНИВЕРСИТЕТ им. В.Г. ШУХОВА»
(БГТУ им. В.Г. Шухова)

Институт информационных технологий и управляющих систем

Кафедра программного обеспечения вычислительной техники и автоматизированных систем

Направление подготовки 09.03.04 Программная инженерия

Направленность (профиль) образовательной программы Разработка программно-информационных систем

Утверждаю:

Зав. кафедрой _____

« _____ » _____ 2026 г.

ЗАДАНИЕ

на выпускную квалификационную работу студента

Пахомова Владислава Андреевича

1. Вид выпускной квалификационной работы (ВКР): *бакалаврская работа.*
2. Тема ВКР *Разработка программного обеспечения для прогнозирования погоды на основе анализа спутниковых снимков с использованием методов машинного обучения* утверждена приказом по университету от «28» января 2026 г. №2/124.
3. Срок сдачи студентом законченной ВКР: «12» июня 2026 г.
4. Исходные данные: ключевые слова, название алгоритмов и методов, технологии разработки и т.п. Пример: рекомендательные системы, подходы построения рекомендательных систем, методы матричной факторизации, оптимизационные методы обучения модели, аффинные преобразования, трёхмерная графика, машинное обучение, клиент-серверное приложение, генеративная нейронная сеть.
5. Содержание ВКР: Здесь должны быть перечислены названия разделов (глав) пояснительной записки. Пример:
 - Введение,
 - Описание предметной области прогнозирования погоды,
 - Разработка архитектуры программного обеспечения прогнозирования погоды,
 - Разработка алгоритмов и программного обеспечения прогнозирования погоды,
 - Тестирование программной системы прогнозирования погоды,
 - Руководство пользователя разработанной системы прогнозирования погоды,
 - Заключение,
 - Список литературы,
 - Приложение А,
 - Приложение Б.

6. Перечень графического материала: 1278 рисунков, 1785 таблиц. Презентация включает: постановку задачи, актуальность, технологии, численные эксперименты, внешний вид программы, заключение.

Консультанты по работе с указанием относящихся к ним разделов:

Раздел	Консультант	Задание выдал (подпись, дата)	Задание принял (подпись, дата)

Дата выдачи задания: «17» января 2026 г.

(подпись руководителя)

Задание принял (приняла) к исполнению

(подпись студента)

КАЛЕНДАРНЫЙ ПЛАН

№ п/п	Наименование этапов работы	Срок выполнения этапов работы	Примечание
1	Анализ предметной области. Постановка задачи разработки системы прогнозирования погоды методами машинного обучения.	18.01.26 – 15.02.26	Выполнено
2	Разработка алгоритмов, структур нейросетей и архитектуры системы прогнозирования погоды.	15.02.26 – 20.03.26	Выполнено
3	Разработка программной части системы прогнозирования погоды.	20.03.26 – 29.03.26	Выполнено
4	Разработка клиентской части системы прогнозирования погоды.	29.03.26 – 29.04.26	Выполнено
5	Тестирование программного обеспечения для прогнозирования погоды.	29.04.26 – 17.05.26	Выполнено
6	Оформление пояснительной записки. Подготовка выступления.	17.05.26 – 10.06.26	Выполнено
5	Тестирование программного обеспечения для прогнозирования погоды.	29.04.26 – 17.05.26	Выполнено
6	Оформление пояснительной записки. Подготовка выступления.	17.05.26 – 10.06.26	Выполнено

Студент (ка) _____ *Пахомов Владислав Андреевич*
(подпись) (Ф.И.О.)

Руководитель ВКР _____ *Осипов Олег Васильевич*
(подпись) (Ф.И.О.)

Результаты проверки ЭВ ВКР на заимствование

Кафедра программного обеспечения вычислительной техники и автоматизированных систем

Студент (ка)	<u>Пахомов В.А.</u>	<u>ПВ-223</u>	<u>16.06.2026</u>
	(Ф.И.О.)	(подпись)	(группа) (дата)

Тема ВКР: Разработка сервиса хранения аудиоданных с применением алгоритмов упаковки и потоковой распаковки.

ВКР прошла проверку на объём заимствований.

Итоговая оценка заимствований: **0%**.

Работу проверил

<u>канд. физ.-мат. наук, доцент</u>	<u>16.06.2026</u>	<u>Хлопов А.М.</u>
(уч. степень, должность)	(дата)	(Ф.И.О.)

Руководитель ВКР

<u>канд. социол. наук, доцент</u>	<u>16.06.2026</u>	<u>Островский А.М.</u>
(уч. степень, должность)	(дата)	(Ф.И.О.)

ОПРЕДЕЛЕНИЯ, СОКРАЩЕНИЯ И ОБОЗНАЧЕНИЯ

РФ — Российская Федерация.

СВО — специальная военная операция.

ВУЗ — высшее учебное заведение.

ВКР — выпускная квалификационная работа.

ИТ — информационные технологии.

БД — база данных.

ПО — программное обеспечение.

СУБД — система управления базами данных.

ООП — объектно-ориентированное программирование.

ЭВМ — электронно-вычислительная машина.

КОП — код операции.

АЛУ — арифметико-логическое устройство.

ПЛИС — программируемая логическая интегральная схема.

БПФ — быстрое преобразование Фурье.

СЛАУ — система линейных алгебраических уравнений.

ОС — операционная система.

API (Application Programming Interface) — программный интерфейс, обеспечивающий взаимодействие между компонентами программного обеспечения.

RGB (red, green, blue) — цветовой формат машинной графики, основанный на смешении компонентов красного, зелёного и синего цветов.

WWW (World Wide Web) — всемирная паутина, система взаимосвязанных гипертекстовых документов.

PDF (Portable Document Format) — переносимый формат электронных документов.

HTML (HyperText Markup Language) — язык гипертекстовой разметки, который используется для создания и структурирования веб-страниц.

SQL (Structured Query Language) — язык структурированных запросов, используемый для работы с базами данных.

VPN (Virtual Private Network) — виртуальная частная сеть, обеспечивающая защищённое соединение через интернет.

ОПРЕДЕЛЕНИЯ, СОКРАЩЕНИЯ И ОБОЗНАЧЕНИЯ							
Изм.	Лист	№ докум.	Подп.	Дата			
Разраб.	Пахомов В.А.		3.06.26	Разработка программного обеспечения для прогнозирования погоды на основе анализа спутниковых снимков с использованием методов машинного обучения	Лит.	Лист	Листов
Руковод.	Осипов О.В.		3.06.26			4	80
Консул.					БГТУ им. В.Г. Шухова, ПБ-223		
Н. контр.	Бондаренко Т.В.		11.06.26				
Зав. каф.	Поляков В.М.		25.06.26				

IoT (Internet of Things) — интернет вещей, концепция подключения различных устройств к интернету для обмена данными.

JSON (JavaScript Object Notation) — текстовый формат обмена данными, который используется для хранения и передачи структурированной информации между системами и приложениями.

					ОПРЕДЕЛЕНИЯ, СОКРАЩЕНИЯ И ОБОЗНАЧЕНИЯ	Лист
Изм.	Лист	№ докум.	Подп.	Дата		5

СОДЕРЖАНИЕ

ВВЕДЕНИЕ	8
1 ОБЗОР ПРЕДМЕТНОЙ ОБЛАСТИ	11
1.1 Актуальность разрабатываемого сервиса	11
1.2 Анализ существующих решений	12
1.2.1 Анализ сервисов хранения аудиоданных	12
1.2.2 Анализ алгоритмов хранения данных	15
1.3 Процесс уменьшения размерности информации	18
1.3.1 Автоэнкодер	18
1.3.2 Обзор архитектуры перцептронов	20
1.3.3 Обзор преобразования входных данных	22
1.3.4 Функции потерь	23
1.4 Постановка задачи	24
1.5 Средства решения задач проекта	26
1.5.1 Язык программирования Python	26
1.5.2 СУБД Postgres	28
1.5.3 Nginx	28
1.5.4 JavaScript	29
1.5.5 Инструмент для контейнеризации Docker	31
1.5.6 NVIDIA Container Toolkit	32
2 Разработка архитектуры программного обеспечения	33
2.1 Архитектура интерфейса взаимодействия с сервером	33
2.1.1 SSR	33
2.1.2 gRPC	34
2.1.3 WebSocket + STOMP	35
2.1.4 REST	35
2.2 Разработка структуры нейронной сети	36
2.3 Проектирование базы данных	40
2.4 Проектирование серверной части приложения	43
2.4.1 Системная архитектура	43
2.4.2 Архитектура приложения	44
2.5 Архитектура фронтенд приложения	49

					Содержание			
Изм.	Лист	№ докум.	Подп.	Дата	Разработка программного обеспечения для прогнозирования погоды на основе анализа спутниковых снимков с использованием методов машинного обучения	Лит.	Лист	Листов
Разраб.	Пахомов В.А.			3.06.26				
Руковод.	Осипов О.В.			3.06.26			6	80
Консул.						БГТУ им. В.Г. Шухова, ПБ-223		
Н. контр.	Бондаренко Т.В.			11.06.26				
Зав. каф.	Поляков В.М.			25.06.26				

2.5.1	Общий слой (Shared)	50
2.5.2	Слой сущностей (Entity)	51
2.5.3	Слой фичей (Features)	51
2.5.4	Слой виджетов (Widgets)	53
2.5.5	Слой страниц (Pages)	53
2.5.6	Слой приложения (App)	53
2.5.7	Архитектура приложения	53
3	Разработка алгоритмов и программного обеспечения	55
3.1	Разработка сервера	55
3.1.1	Слой хранения данных	55
3.1.2	Валидация входных и выходных параметров, схемы	63
3.1.3	Сервисы	66
3.1.4	Внешний интерфейс	69
3.1.5	Авторизация	71
	ПРИЛОЖЕНИЕ А Техническое задание	74
A.1	Введение	74
A.2	Основания для разработки	74
A.3	Назначение разработки	74
A.4	Требования к программе или программному изделию	74
A.4.1	Требования к функциональным характеристикам	74
A.4.2	Требования к надёжности	76
A.4.3	Условия эксплуатации	76
A.4.4	Требования к составу и параметрам технических средств	76
A.4.5	Требования к информационной и программной совместимости	76
A.4.6	Требования к маркировке и упаковке	76
A.5	Требования к программной документации	76
A.6	Технико-экономические показатели	77
A.7	Стадии и этапы разработки	77
A.8	Порядок контроля и приёмки	77
	ПРИЛОЖЕНИЕ Б Листинги разработанных схем для создания композиции	78

ВВЕДЕНИЕ

Мир современного человека всё больше переходит в цифровое пространство, и это не удивительно. Цифровое пространство обладает неоспоримыми преимуществами при работе с информацией:

- Дешёвое хранение информации, а значит её большой объём
- Большая скорость передачи информации
- Возможность обрабатывать большое количество информации очень быстро
- Качество хранимой информации

Эти факторы так или иначе и стали решающими в решении хранения данных как в быту человека, так и в бизнес-процессах и в творчестве.

Самым ярким и приближённым к цифровому пространству примером является профессия программиста. На это влияет как сама специфика и предназначение профессии, так и вышеописанные процессы. Можно ли себе представить программиста, который совсем не владеет компьютером? Конечно же нет. Возьмут ли на работу программиста, который продолжает хранить весь свой код на листах бумаги? Только если эта компания почему-то любит запускать очень старые ракеты. Современного конкурентноспособного разработчика сложно представить без умных текстовых редакторов, хранения кода в облаке, автоматизации процессов доставки клиентам, нейронных сетей помогающих в написании кода (а иногда ещё и пишущих за них).

Сама профессия по личному мнению автора лежит на стыке творческого процесса и бизнес процессов в разной степени важности в зависимости от разрабатываемого продукта. Она требует как изобретательности, креативности и нестандартных подходов при решении различных задач, так и соответствия конкретным рамкам, условиям и отлаженным процессам.

На примере профессии программиста можно увидеть, насколько сильно может интегрироваться цифровое пространство в сферу требующую как творческого подхода, так и точности, и насколько много пользы оно может принести при решении задач. Важно, однако, отметить, что цифровое пространство всё ещё не может полностью вытеснить другие подходы. Это такой же инструмент, как, например, молоток. Оно решает конкретные проблемы, связанные преимущественно с информацией.

Одна из сфер, которая очень похожа на программирование, является музыкальная сфера. Она также требует творческого подхода, но также задаёт в себе

					ВВЕДЕНИЕ			
Изм.	Лист	№ докум.	Подп.	Дата	Разработка программного обеспечения для прогнозирования погоды на основе анализа спутниковых снимков с использованием методов машинного обучения	Лит.	Лист	Листов
Разраб.	Пахомов В.А.			3.06.26				
Руковод.	Осипов О.В.			3.06.26			8	80
Консул.						БГТУ им. В.Г. Шухова, ПБ-223		
Н. контр.	Бондаренко Т.В.			11.06.26				
Зав. каф.	Поляков В.М.			25.06.26				

определённые рамки.

И да, за последнее время интеграция цифрового пространства в этой сфере тоже сильно продвинулась. Сервисы музыкального стриминга, согласно исследованиям, используют уже более трети россиян, а рост аудитории продолжается. В процессе написания музыки всё чаще используются DAW, умеющие работать с продвинутым оборудованием записи; синтез звука и его обработка тоже проходили очень долгий путь эволюции.

Однако на рынке продюсирования музыки почему-то очень мало инструментов, которые помогают в разработке аудиоданных. Множество продюсеров всё ещё складывают свои проекты в крайне неорганизованное хранилище часто представляющее собой обычную папку на локальных персональных компьютерах. А если данные будут утеряны, то обычно с этим ничего поделать нельзя и их приходится восстанавливать "по памяти".

И такие проблемы очень легко решить. Тем более, для них уже есть решение. И их можно позаимствовать из той же ранее рассмотренной профессии программиста. Облачное хранилище данных позволит избежать проблемы утери данных а также позволит делиться ими касанием пальца, что тоже повысит эффективность общения с коллегами. Хранение предыдущих версий позволит возвращаться к старым концепциям, пересматривать их, рефлексировать и адаптировать в разработке. Распространение данных в централизованной платформе гораздо более удобно для конечного потребителя в лице дистрибьютора или слушателя. Дополнение платформы в виде бизнес-сущностей позволит дистрибьюторам при публикации полученных композиций.

Создание единой платформы также создаёт множество задач по обработке и выдаче данных. Аудиоданные могут быть очень большими, особенно если это демонстрационные записи, не отличающиеся особой лаконичностью. В связи с этим должна быть также возможность порционной выдачи данных в реальном времени.

Первая глава посвящена анализу предметной области, в ней рассматривается актуальность создания сервиса, выполняется сравнительный анализ готовых решений, формируются функциональные и нефункциональные требования к разрабатываемому сервису.

Вторая глава описывает проектирование: общие архитектурные паттерны, рассмотрена клиент-серверная составляющая и интерфейсы их взаимодействия, архитектура хранилищ данных

Третья глава содержит структуру программного обеспечения: модели и серви-

					ВВЕДЕНИЕ	Лист
						9
Изм.	Лист	№ докум.	Подп.	Дата		

сы, конкретные библиотеки при реализации.

Четвёртая глава описывает инфраструктуру сервиса: процессы контейнеризации, выдачи статических данных, конфигурации Nginx и обратного проксирования, подключения графических устройств для работы алгоритма сжатия.

Разрабатываемый сервис соответствует концепциям и тенденциям развития отрасли, поддерживаемый и расширяемый. Цель реализуемого сервиса - управление процессами разработки композиции а также упрощение её распространения в профессиональной среде.

					ВВЕДЕНИЕ	Лист
						10
Изм.	Лист	№ докум.	Подп.	Дата		

1 ОБЗОР ПРЕДМЕТНОЙ ОБЛАСТИ

1.1 Актуальность разрабатываемого сервиса

Разработка музыкальных композиций занимает одно из важных мест на сегодняшнем рынке. Количество потребителей музыкального контента каждый день продолжает расти, о чём говорит статистика музыкальных стриминговых сервисов. Однако инструментов для управления разработки и обеспечения её эффективности у современных продюсеров представлено очень мало.

На текущий момент самым распространённым способом хранения аудиоданных является их хранение на локальных носителях или же на хранителе персональных компьютеров. Повреждение или утеря хранилищ приведёт к утере данных а также к необходимости их восстановления с нуля, что будет занимать очень много времени а также имеет риск привести к невозможности восстановления изначально-го варианта ввиду несовершенства человеческой памяти.

Важным аспектом является развитие композиции. Один проект или же композиция могут иметь несколько различных форм. Например, может измениться инструментальная или вокальная партия. Кроме того, отслеживание предыдущих версий позволит найти более удачный вариант исполнения и использовать наработки в новой итерации. Также вид композиции может сильно разниться в зависимости от его целей и вариантов исполнений. В отношении живой музыки это может быть композиция записанная в живом исполнении или в студии. Путём введения запрета на редактирование и удаление данных можно обеспечить как улучшенные гарантии хранения данных, так и сохранение всех версий композиции.

Для возможности отделения музыкальных композиций по их назначению можно ввести понятие тегов. Тег - короткое строковое описание версии, которое описывает цель этого релиза.

При условии наличия большого количества версий композиции возникает вопрос составления серий композиций, или же плейлистов. Так как каждый из треков имеет свою определённую ценность и форму, плейлист так же может иметь свою ценность. Например, владельцу площадки для выступлений будет не очень интересно услышать, как группа будет звучать в живой записи, гораздо большую ценность для него будет предоставлять живое выступление. Плейлисты позволяют быть гораздо более гибким и предоставлять различным заказчикам интересную для

					ОБЗОР ПРЕДМЕТНОЙ ОБЛАСТИ			
Изм.	Лист	№ докум.	Подп.	Дата	Разработка программного обеспечения для прогнозирования погоды на основе анализа спутниковых снимков с использованием методов машинного обучения	Лит.	Лист	Листов
Разраб.		Пахомов В.А.		3.06.26				
Руковод.		Осипов О.В.		3.06.26			11	80
Консул.						БГТУ им. В.Г. Шухова, ПБ-223		
Н. контр.		Бондаренко Т.В.		11.06.26				
Зав. каф.		Поляков В.М.		25.06.26				

его задач информацию.

Предоставление служебной информации также сильно облегчит процесс взаимодействия с проектом. В любой момент можно посмотреть название проекта, исполнителей, описание без дополнительного обращения, что позволит увеличить эффективность разработки.

Важным аспектом является налаживание коммуникации между командой разработчиков. Самым простым способом может быть система комментирования, позволяющая оставить наработки, мысли и отзывы команды разработки касательно композиции.

Возможность запустить сервис на собственных серверах может стать решающей для большинства компаний для обеспечения в первую очередь конфиденциальности информации.

Введение запрета на редактирование и удаление композиций неизбежно ведёт к использованию большого объёма памяти. Необходимо выбрать алгоритм, кодек или же разработать собственный подход для хранения аудиоданных.

Таким образом, разрабатываемый сервис позволит получить инструмент, обеспечивающий централизованное защищённое хранилище наработок в музыкальной сфере, позволяющее наладить рабочие процессы внутри команды а также наладить общение с заказчиками.

1.2 Анализ существующих решений

1.2.1 Анализ сервисов хранения аудиоданных

Готовых продуктов, обеспечивающих описанную логику, очень немного, вот несколько самых популярных из них:

- Git
- Splice
- DropBox
- Boombox.io

Проведём сравнительный анализ.

Git, хорошо известный в среде программирования, не является программным обеспечением, направленным именно на хранение аудиоданных или любых других больших медиа данных. Он лучше всего справляется именно с текстовой информацией. Кроме того, Git - это система контроля версий, а не отдельный сервис. Он хранит историю об изменениях, но не имеет привязки к какому-либо сервису. Это же и преимущество, так как декомпозиция на сервис и программу позволяет

					ОБЗОР ПРЕДМЕТНОЙ ОБЛАСТИ	Лист
Изм.	Лист	№ докум.	Подп.	Дата		12

публиковать свои данные на любой совместимый продукт, вроде GitHub, GitLab, BitBucket или же развёрнутый самостоятельно сервис. Систему плейлистов можно попробовать воссоздать внутри папки, а значит по возможности распространения он сильно уступает. Git обеспечивает надёжное хранение данных, использует концепцию невозможности удаления и изменения данных, очень надёжен и имеет систему тегов. Сервисы позволяют наладить отличную работу в команде с использованием запросов на слияние, комментариев. Git и сервисы невероятно гибки, но они дают слишком много свободы и предлагают слишком мало удобств для данной задачи.

Splice на сегодняшний день отошёл от концепции хранения версионных аудиоданных и стал скорее сервисом, содержащим сэмплы - короткие музыкальные фразы. Ранее сервис предлагал облачное хранилище для проектов в DAW и коллаборативную работу над ними в облачном пространстве. Всё ещё нет системы плейлистов и обозначения композиций и нет возможности развернуть собственное хранилище.

DropBox - этот вариант чуть лучше, чем хранение на диске. Как минимум, данные уже содержатся в облаке, а значит к ним есть доступ. Существуют инструменты для интеграции с DAW. Но это нельзя назвать полноценным сервисом для хранения аудиоданных. Это просто сервис хранения данных в облаке, который через плагины используется для хранения аудио. DropBox поддерживает создание тегов и версионирования файлов, создания своих свойств для файлов. Но создания обновляемых плейлистов всё ещё нет. Эта система не заточена именно под хранение аудиоданных.

Boombox.io - самый близко подобранный к требованиям сервис. Он умеет создавать версии треков, приватные плейлисты. Композиции снабжаются метаданными а также в нём много инструментов ИИ. Существует комментирование с указанием временных меток. Однако данные изменяемые - это значит, что хранимые они могут быть нарушены. Сервис находится исключительно в облаке и не может быть предоставлен кампаниям в виде сервиса для собственного развёртывания.

Таблица 1.1 – Сравнительная таблица сервисов хранения аудиоданных

	Git	Splice	DropBox	Boombox.io
Хранение данных в облаке	Зависит от сервиса	Есть, облачный сервис	Есть, облачный сервис	Есть, облачный сервис

Продолжение таблицы 1.1

	Git	Splice	DropBox	Boombox.io
Версионирование и тегирование	Есть, полная система	В данный момент устарела, отсутствует	Есть 180-дневная история изменений по платной подписке	Присутствуют версии, нет тегов
Инструменты коллаборации	Зависит от сервиса (комментарии, запросы на слияние и пр.); Отдельные ветки и слияние веток	Совместная работа в DAW	Совместная работа над файлами, комментирование	Комментарии с указанием таймкодов, обычные комментарии
Поддержка разворачивания собственного сервиса	Есть	Нет, облачный сервис	Нет, облачный сервис	Нет, облачный сервис
Создание плейлистов с указанием версий	Возможно, но неудобно	Нет	Возможно, но неудобно	Есть
Указание метаданных	Выполняется вручную - неудобно	Есть	Есть, нестандартизировано	Есть, выполняется полуавтоматически
Неизменность данных	Есть	Нет	Нет	Нет

Продолжение таблицы 1.1

	Git	Splice	DropBox	Boombox.io
Оптимизации для хранения аудиоданных	Выполняются вручную - неудобно	Не имеет значения	Выполняется вручную	Нет

Проведённое исследование показывает, что сервисы можно разделить на две категории.

Первая категория - это сервисы предназначенные для работы исключительно с одной конкретной задачей. Git решает вопрос версионирования, а DropBox решает проблему хранения в облаке. Обычно работа с такими инструментами требует дополнительной обработки, оптимизации и введения правил, по которым они будут храниться. Они предоставляют решение одной конкретной задачи и потому слишком универсальны, они не предоставляют оптимизаций для работы конкретно с аудио данными.

Вторая категория - сервисы, направленные непосредственно на работу с аудио. Начнём с того, что они крайне непопулярны. Их очень мало, и поэтому, чтобы продолжать выживать, они инкапсулируют свои собственные решения на своих серверах и предлагают их за плату. Такие решения могут быть угрозой конфиденциальности для бизнеса.

Моё решение будет брать лучшее из двух категорий. В нём можно найти специфичный для аудиоданных и их распространения функционал, а также решённые задачи версионирования и хранения данных в облаке. Кроме того, текущее решение распространяется в свободном доступе.

1.2.2 Анализ алгоритмов хранения данных

Алгоритмы кодирования аудиоданных прошли очень долгий путь развития, рассмотрим несколько из них

- MP3
- Opus
- FLAC
- WAV

MP3 - аудиокодек с потерей данных. Этот кодек использует психоакустические оптимизации для того, чтобы сократить количество занимаемого места. Из-за этих

					ОБЗОР ПРЕДМЕТНОЙ ОБЛАСТИ	Лист
Изм.	Лист	№ докум.	Подп.	Дата		15

оптимизаций, особенно на маленьком битрейте, может возникать очень множество артефактов при прослушивании. Звук становится наиболее чистым и неотличимым от исходного при 320 КБит/с. Однако, даже при большом битрейте, кодек всё ещё допускает потери данных, что недопустимо в профессиональной сфере.

Opus - современный кодек, используемый в основном для стриминга ввиду его очень маленького битрейта (около 96 КБит/с) при котором он становится неотличим от оригинального звука. Недостатком является то, что всё ещё очень мало устройств могут поддерживать этот формат, а также он даёт большую нагрузку для устройства воспроизводящего аудио. Кроме того, кодек тоже допускает потерю данных, что может быть заметно на хорошем оборудовании. Кодек Opus оптимизирован под одну конкретную частоту дискретизации - 48000 Гц, что так же ограничивает его использование в студиях.

FLAC - старый формат без потери данных. Он использует модифицированный алгоритм кодирования Хаффмана - кодирование сдвига Хаффмана. В фрейм записываются не повторяющиеся данные, а их линейный рост. Такой метод сжатия позволяет достичь коэффициента сжатия в 2 раза, что меньше чем предыдущие кодеки, однако и потери данных в нём нет.

WAV - кодек, содержащий необработанный уровень амплитуды без сжатия. Практически не влияет на загруженность устройства, однако и содержит в себе больше всего данных. Идеально подходит для записи и использования в DAW ввиду простоты его распаковки, но плох для длительного хранения.

Таблица 1.2 – Сравнительная таблица кодеков

	MP3	Opus	FLAC	WAV
Коэффициент сжатия	10	17	1.7	Без сжатия
Нагрузка при кодировании/декодировании	Средняя	Высокая	Низкая	Крайне низкая

Продолжение таблицы 1.2

	MP3	Opus	FLAC	WAV
Поддержка кодеков на устройствах	Очень хорошая поддержка на старых и новых устройствах	Работает на- тивно толь- ко на отно- сительно но- вых устрой- ствах	Хорошая поддержка на старых и новых устройствах	Может пло- хо работать на опера- ционных системах кроме Windows, в остальном довольно хорошая поддержка
В восстано- вленных дан- ных нет по- терь	Нет	Нет	Да	Да
Исполь- зование в стриминге	Хорошо под- ходит для стриминга	Лучше всего под- ходит для стриминга	Редко ис- пользуется в стриминге, только там где важно качество	Требуется очень боль- шого про- пускного канала для стриминга
Битрейт	320 КБит/с	96 КБит/с	800 КБит/с	1411 КБит/с
Влияние на исходный материал	Сильное ис- пользование психоаку- стических принципов	Сильное ис- пользование психоаку- стических принципов	Нет	Нет

Кодеки MP3 и Opus подходят для хранения аудиоданных для конечного потре-
бителя, так как включают в себя психоакустические приёмы: затухание, удаление
неслышимых данных, маскирование. При дополнительной обработке данных в DAW
утраченные данные могут повлиять на результат выполняемой работы, поэтому для
текущей задачи эти кодеки использовать можно, но только в определённых сценари-

ях, вроде подготовки выпуска композиции в финальных сервисах дистрибуции.

Кодеки FLAC и WAV же не искажают исходные данные, а значит они лучше подходят для композиций записанных в данном формате. Но их узким местом является количество занимаемых данных. FLAC частично решает эту проблему используя математические методы кодирования вместо психоакустических. Однако степень сжатия данных в особенно сложных местах может быть достаточно маленькой.

Можно модифицировать алгоритм FLAC и предоставить возможность кодировать особо сложные фреймы при помощи методов, допускающих потерю данных, однако не влияющих на исходную волну амплитуды достаточно сильно.

1.3 Процесс уменьшения размерности информации

Процессы уменьшения размерности информации это задачи которые позволяют сократить количество свойств внутри какого-либо объекта за счёт нахождения связи между свойствами этого объекта.

Существует множество способов, позволяющих сократить размерность данных. Математические способы обычно используют матрицу корреляции для определения взаимосвязи между свойствами внутри объекта. По этой же матрице данные можно восстановить обратно.

Однако аудиоданные часто могут содержать более комплексные связи, которые невозможно выразить при помощи таких матриц. В небольшом отрезке из аудиофайла могут быть как сильные всплески амплитуды, так и постоянные данные. Однако музыкальная композиция всё ещё подчиняется гармоническим частотным правилам, соотношениям частот внутри аккордов, темпу и ритму.

Для нахождения неочевидных и сложных закономерностей всё чаще применяются свёрточные нейросети (CNN).

1.3.1 Автоэнкодер

Автоэнкодер по своей структуре очень похож на обычную свёрточную нейросеть. Он так же состоит из перцептронов, у него есть скрытые слои, входной и выходной слои. Однако есть в архитектуре и различия.

Автоэнкодер можно разделить на две составляющие: энкодер и декодер. Задача энкодера - уменьшить размерность данных. Каждый из последующих слоёв уменьшает размерность данных. Получившийся вектор называют латентным вектором или же латентом. Он содержит в себе только самую важную информацию. Входным вектором для декодера как раз будет являться латентный вектор. Задача декодера - восстановить пробелы в латенте чтобы получить исходный объект. Если энкодер

шёл на уменьшение количества свойств объекта, то декодер уже увеличивает их количество в обратном порядке.

Чем меньше разрыв между количеством перцептронов в каждом слое, тем более качественным должен получиться результат упаковки/распаковки. Однако с увеличением количества перцептронов нейросеть будет обучаться гораздо дольше, проход будет выполняться дольше а ресурсов обеспечивающих обучение и использование нейросети может не хватить. На рисунке 1.1 изображен пример структуры автоэнкодера.

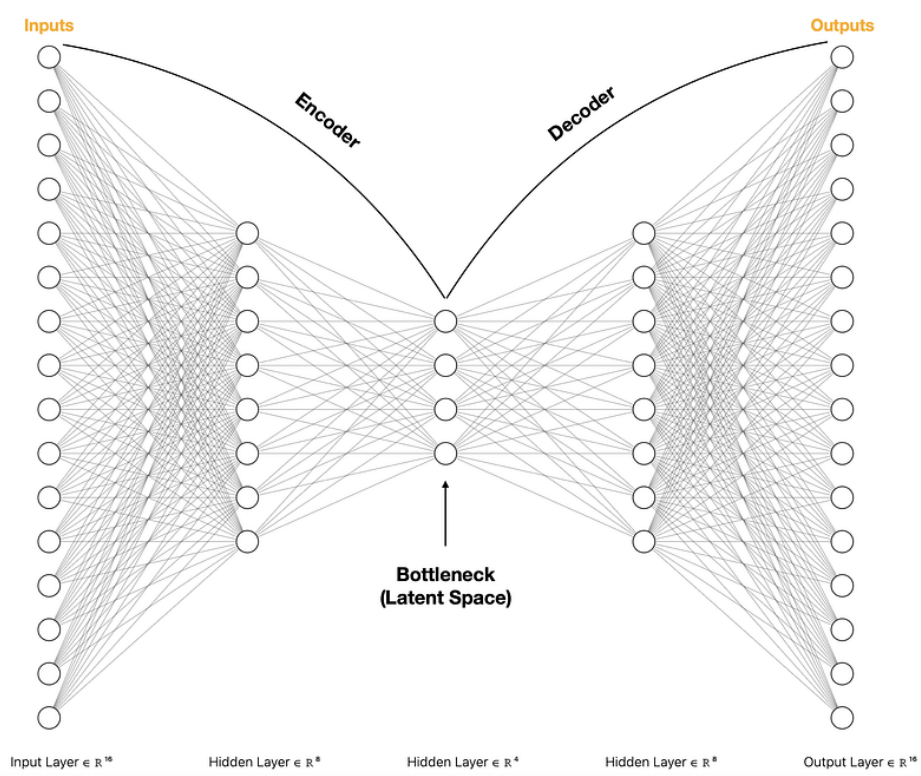


Рисунок 1.1 – Пример структуры автоэнкодера [1]

Автоэнкодеры находят своё применение не только в задачах уменьшения размерности вектора, но и в задачах преобразования исходных данных. Например, удаление шума с изображения, как показано на рисунке 1.2.

У данной архитектуры нейронной сети для текущей задачи есть как положительные, так и отрицательные стороны. Часто для восстановления музыкального отрезка очень важен контекст: что звучало до текущего отрезка, так как контекст содержит информацию о волнах. Некоторые особенно низкие частоты могут быть и не определены как частота в виду достаточно маленького времени. Минимально слышимая для человеческого уха частота в 20 Гц потребует около 0,05 секунд для проигрывания одного периода. Проблему контекста в данной архитектуре можно решить при помощи увеличения фрейма, например, до 1 секунды. Этой информа-

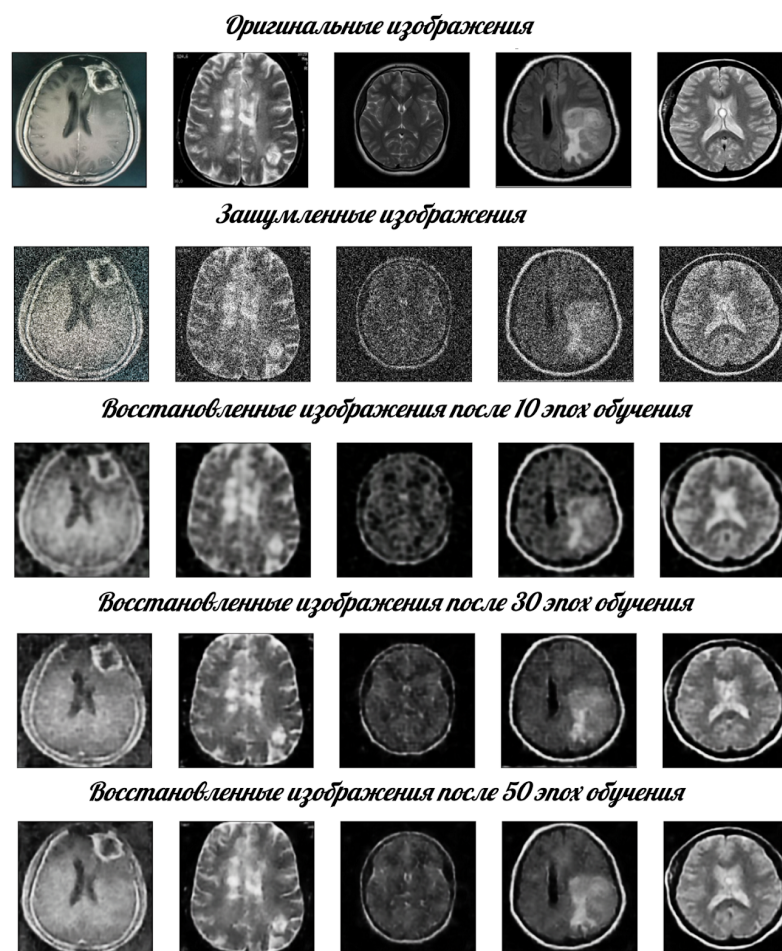


Рисунок 1.2 – Удаление шума при помощи автоэнкодера [2]

ции будет достаточно для сохранения частот. Однако с увеличением входного слоя увеличивается сложность обучения, требования к аппаратному обеспечению растут.

Есть и плюсы в использовании текущего подхода. Декодирование отдельных независимых друг от друга фреймов позволит масштабировать процесс декодирования при появлении более продвинутого аппаратного обеспечения путём параллельного декодирования. Кроме того, автоэнкодер довольно просто обучать и использовать.

1.3.2 Обзор архитектуры перцептронов

Как уже было сказано ранее, автоэнкодер может быть достаточно тяжёлой свёрточной нейросетью при использовании большого количества входных параметров. Существуют способы оптимизации, позволяющие сократить количество связей между нейронами.

Полносвязная нейросеть

Полносвязный слой строится по принципу "нейрон связан с каждым нейроном из предыдущего слоя". В контексте аудиоданных это может быть как положительным

примером, так и отрицательным. Так, например, более низкие частоты с большим периодом смогут влиять на значение текущего нейрона, а значит информация о низких частотах будет сохранена. Но полносвязные слои крайне неоптимизированны. Вместе с важными данными в нейрон может приходить и ”мусор” который будет только добавлять шума в декодированной записи.

При использовании полносвязной нейросети требуется тонкая настройка входного слоя. Далеко не все данные могут быть полезными, и слишком большой входной вектор может привести к очень большому времени достижения экстремума для нейронной сети при обучении.

Использование Convolution слоёв

Convolutional слои позволяют создавать неполносвязную нейронную сеть. На примере одномерного массива Conv-слой сможет выбирать лишь небольшой участок из более большого вектора. Кроме того, Convolution слои могут работать как с одномерными, так и с двумерными, трёхмерными слоями. Например, на рисунке 1.3 изображён пример поведения Convolutional слоя. Он не использует при вычислении все входные параметры, а выбирает только соответственно своему фильтру

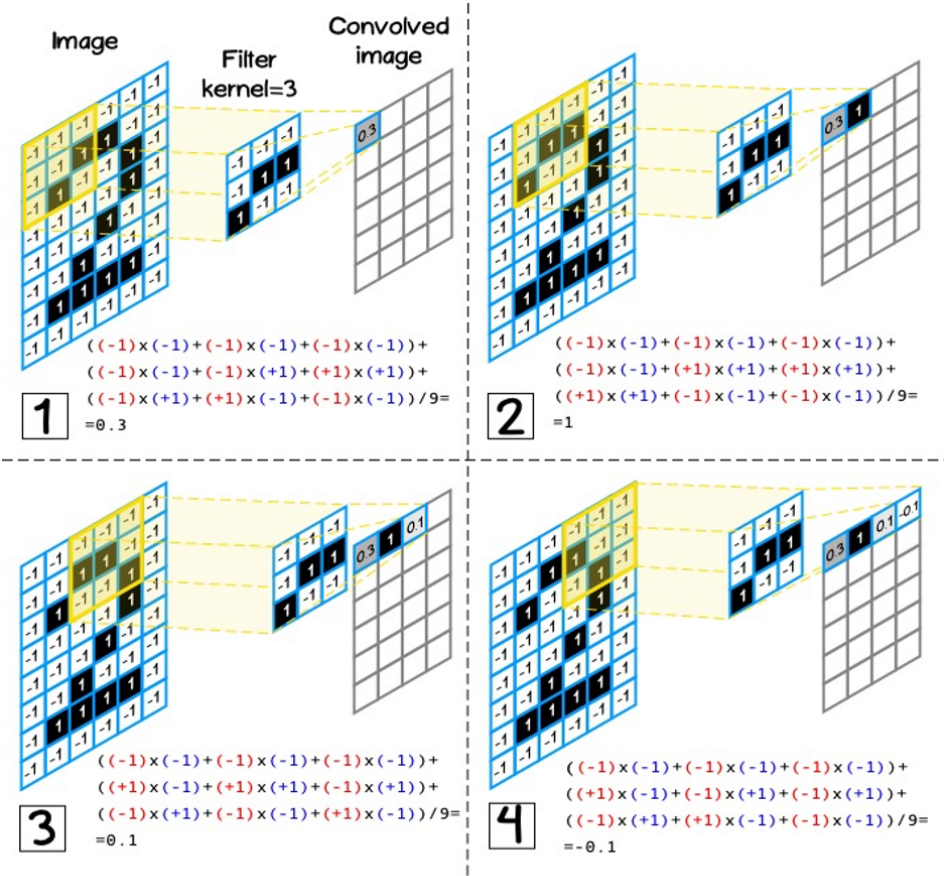


Рисунок 1.3 – Принцип работы Convolutional Layer [3]

Использование Convolutional слоёв позволяет решить проблему слишком боль-

шой выборки данных, что позволит уменьшить количество связей между нейронами. Уменьшение связей между нейронами позволит увеличить размер входного слоя, что позволит увеличить количество кодируемой/декодируемой информации за один раз. Однако при использовании такого рода слоёв всё ещё остаётся проблема настройки выборки. Кроме того, при потоковой распаковке аудио гораздо более ценна работа с относительно небольшим фреймом данных.

1.3.3 Обзор преобразования входных данных

Нейронные сети умеют работать с данными различных размерностей. Рассмотрим, в какие форматы данных можно было бы преобразовать аудиоданные.

Одномерный вектор

Это самый простой способ. Аудиоданные преобразуются в одномерный массив, свойство каждого из которых содержит амплитуду. Этот способ относительно лёгок в реализации и не требует больших преобразовательных мощностей, так как фактически выполняется преобразование аудиоданных в последовательность амплитуд, что естественно для большинства аудиоформатов на сегодняшний день, и производится дополнительная нормализация данных для работы в нейронной сети.

Двухмерная спектрограмма

Можно преобразовать аудиоданные в более сложный и требующий больших вычислений формат. Спектрограмма представляет собой изображение. Одна из координат может представлять собой время, а другая - частоту. Яркость точки на изображении в точке (x, y) будет показывать, насколько сильна амплитуда частоты x в момент y. Для получения такого изображения используются алгоритм быстрого преобразования Фурье. Для этого способа так же требуется достаточно большой размер фрейма. Если фрейм будет недостаточно большим, то алгоритм просто "не услышит" длинные волны. На рисунке 1.4 представлена спектрограмма записи голоса человека, говорящего Hello World, после чего он кашляет. Видно по первому всплеску, что у голоса есть определённый тон и повторяющиеся гребни - октавы. Кашель же - это в основном шум, поэтому основной тон там не так заметен.

Использование такого подхода так же будет вызывать трудности и с декодированием спектрограммы. Для этого тоже используются специальные алгоритмы Гриффина-Лима или обратного быстрого преобразования Фурье. Данный способ даёт большую нагрузку на аппаратное обеспечение и может вызывать задержки при потоковой распаковке.

					ОБЗОР ПРЕДМЕТНОЙ ОБЛАСТИ	Лист
Изм.	Лист	№ докум.	Подп.	Дата		22

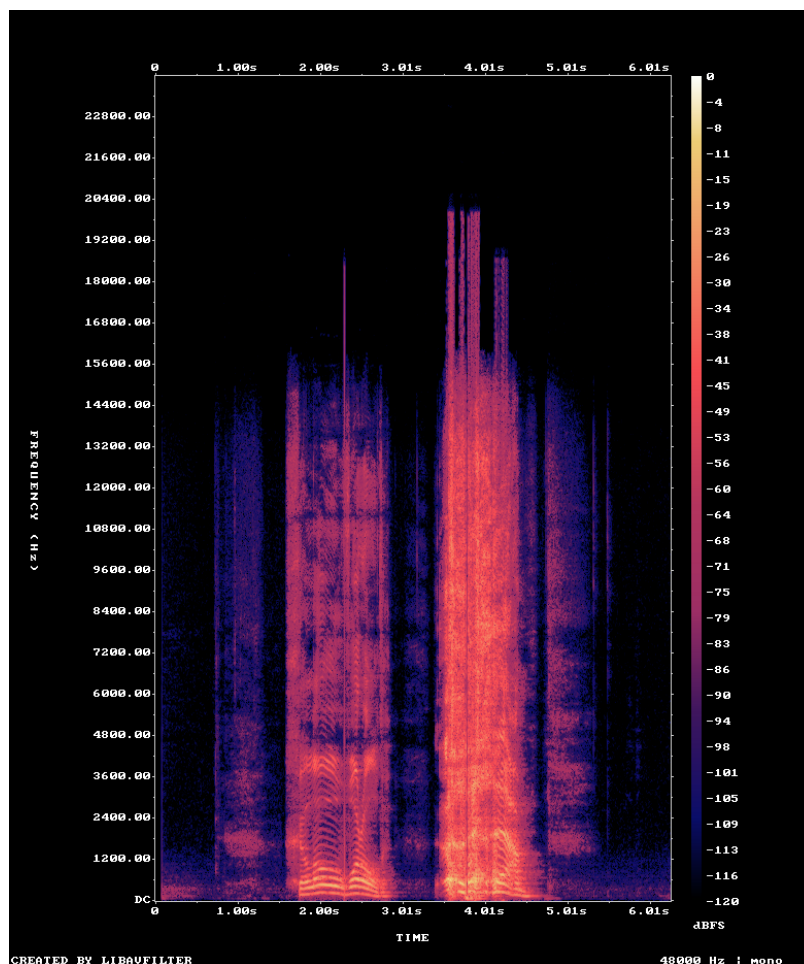


Рисунок 1.4 – Запись голоса человека в спектрограмме, авторский материал

1.3.4 Функции потерь

Функции потерь будут прямо влиять на процесс обучения и, соответственно, на получившееся звучание аудиоданных. Именно функция потерь определяет то, на что будет сфокусировано внимание при обучении.

MSELoss

Самая простая функция. Предположим, что у нас есть исходный сигнал y и предсказанный сигнал $\bar{y}$ размерность N . Тогда функция потерь MSE будет высчитана как

$$MSE = \frac{1}{N} \sum_{i=1}^N (y_i - \bar{y}_i)^2$$

Эта функция может быть не очень хороша для аудиоданных, так как она слабо учитывает отклонения во всплесках - в аудио это может быть удар бочки или же по тарелке. Но она неплохо восстанавливает общую картинку и общую слышимость.

Spectral Loss

Функция спектральной потери считает потери по определённым частотам. Эта функция потерь гораздо более близка к природе аудиоданных, так как работает с её частотами напрямую. Обычно высчитывается потеря по нескольким частотам отра-

жающим одну ноту, например [128 Гц, 256 Гц, 512 Гц, 1024 Гц]. Для определения амплитуд частот используется быстрое преобразование Фурье. Эта функция потерь гораздо лучше сохраняет детали, нежели общее звучание. Семейство функций спектральной потери часто используется при обучении нейронных сетей, работающих с аудиоданными.

Положим, что у нас есть исходный сигнал y и $\bar{y}$. Тогда функция спектральной потери будет высчитана следующим образом

$$SpectralLoss = |\log(|STFT(y)| + \epsilon) - \log(|STFT(\bar{y})| + \epsilon)|$$

Где ϵ - очень маленькая величина.

Чаще всего используется комбинированная функция потери в зависимости от цели при обучении.

1.4 Постановка задачи

На основе представленной информации можем сформировать список желаемого функционала в сервисе

- Аутентификация и регистрация для управления бизнес сущностями.
- Возможность работы с авторами композиции.
 - Автор должен содержать имя, аватар, описание
 - Должны поддерживаться операции CRUD: добавление, удаление, редактирование и получение
 - Необходимо предоставить графический интерфейс для работы с указанными операциями
- Возможность работы с группами
 - Группа - это объединение авторов. Она должна включать в себя аватар, описание, участников
 - Должны поддерживаться операции CRUD: добавление, удаление, редактирование и получение
 - Необходимо предоставить графический интерфейс для работы с указанными операциями
- Возможность создавать песню
 - Песня должна содержать в себе справочную информацию: имя, описание, тэг, BPM, тональность, слова, авторов и группы а также файлы этой песни.
 - Должна быть возможность указать ведущих файлов. Эти файлы репрезентируют песню. Так, ведущее изображение является картинкой этой песни. А ведущее аудио - проигрывающей песней при её запуске. Дополнительные файлы могут содержать прочую важную информацию.

					ОБЗОР ПРЕДМЕТНОЙ ОБЛАСТИ	Лист
Изм.	Лист	№ докум.	Подп.	Дата		24

- После создания песню удалить нельзя - можно только выпустить следующий релиз песни. Всегда можно просмотреть предыдущие релизы песен.
- Должен быть доступен чат внутри каждой из песен, где пользователи могут просматривать а также отправлять свои собственные сообщения
- Должна быть реализована возможность просмотра конкретной версии песни.
- Должны быть реализованы операции в API для выполнения вышеобозначенных операций.
- Необходимо предоставить графический интерфейс для работы с указанными операциями.
- Плейлист
 - Плейлист - это сущность, которая собирает в себе песни и фильтрует их по фильтру. Плейлист должен в себя включать изображение, название и описание.
 - Функция фильтрации заключается в поиске по текстовому полю, который может представлять собой либо название тега либо название версии. Если указано название тега, в песне должна быть найдена наиболее новая песня с указанным тегом. Если фильтр не указан, будет предоставлена последняя версия песни. Если же указан номер версии, будет найдена версия с этим конкретным номером.
 - Должны быть реализованы операции CRUD: добавление, удаление, редактирование и получение
 - Должен быть предоставлен графический интерфейс для взаимодействия с указанными операциями.
- Взаимодействие с файлами
 - Должен быть предоставлен альтернативный способ хранения аудиоданных, использующий доверенный кодек для хранения аудиоданных.
 - Необходимо предоставить возможность скачивать загруженные файлы
 - Отправка аудиоданных со своим кодеком должна поддерживать специальные заголовки, позволяющие получать данные порционно. Распаковка не должна происходить одномоментно и должна распаковываться постепенно.
- Проигрыватель
 - Должен быть реализован проигрыватель, позволяющий проиграть плейлист или же отдельную версию песни.
 - Плейлист должен поддерживать следующие операции: выбрать порядок в

					ОБЗОР ПРЕДМЕТНОЙ ОБЛАСТИ	Лист
						25
Изм.	Лист	№ докум.	Подп.	Дата		

пределах плейлиста (перемешать, в обычном порядке, зациклить), выбрать порядок в передлах песни (зациклить песню или разрешить переход к следующим), выбрать следующий трек, выбрать предыдущий трек, отразить информацию о песне, выбрать громкость.

- Должно быть реализовано API достаточное для выполнения описанных функций
- Должен быть реализован графический интерфейс, поддерживающий эти операции

В соответствии с функциональными требованиями а также целью ВКР поставим задачи:

1. Разработка или поиск методов оптимизированного хранения аудиоданных.
2. Разработка архитектуры и программного обеспечения для хранения аудиоданных.
3. Тестирование разработанного программного обеспечения для хранения аудиоданных.
4. Создание руководства пользователя разработанного программного обеспечения для хранения аудиоданных.

1.5 Средства решения задач проекта

1.5.1 Язык программирования Python

Python - интерпретируемый язык программирования, разработанный и представленный в 1991 году. Python изначально планировался как язык программирования, очень похожий на язык псевдокода. Именно поэтому в нём не получится найти фигурные скобки, точки с запятыми. Исходя из этого подхода Python унаследовал не только лаконичность, но и также строгие синтаксические правила оформления кода (PEP8), без которых он не запустится.

Ввиду своей простоты язык программирования стал очень популярен в различных сферах как прототипирования, так и написания сложных серьёзных программ. Сейчас язык используется для описания прямых высокоуровневых команд и действий, будь это эндпоинт веб-сервера или же обращение к видеокарте. Это стало возможно благодаря тому, что большинство библиотек Python являются интерфейсами к исполняемым файлам, содержащим основную логику и ценность библиотеки.

Такой подход позволяет управлять сложной большой машиной через относительно лёгкий и понятный человеку интерфейс. Для поставленных задач язык программирования предоставляет специализированные инструменты.

					ОБЗОР ПРЕДМЕТНОЙ ОБЛАСТИ	Лист
Изм.	Лист	№ докум.	Подп.	Дата		26

Для работы с нейросетями предоставляется пакет Torch - это библиотека позволяющая создавать и обучать нейронные сети, умеющая работать с аппаратным обеспечением через интерфейсы CUDA, ROCm, Metal. Для работы с математическими вычислениями, например, при вычислении функции потерь можно использовать популярную библиотеку NumPy, предоставляющую широкий спектр математических функций и преобразований.

В качестве библиотеки, позволяющей развернуть свой собственный REST API веб-сервер, можно взять FastAPI. Эта библиотека предоставляет возможность быстрого создания точек обращения к интерфейсу взаимодействия с сервером с использованием системы Dependency Injection - внедрения зависимостей, позволяющей подключать к обслуживаемой точке только необходимые компоненты, будь то сервис или репозиторий.

Для контроля входных параметров а также обеспечения их валидности для Python предоставляется библиотека Pydantic, действующая по принципу Parse Don't Validate - паттерну, согласно которому каждый ввод обязательно должен быть преобразован в более высокоуровневые структуры данных обеспечивающих корректность данных. Чёткие правила валидации для каждого поля в API позволяет избежать возможных проблем с безопасностью в виде инъекций. Библиотеки семейства Pydantic также предлагают удобный инструментарий для работы с переменными среды и настройками проекта, установке фильтров и сортировок для выборки.

Для взаимодействий с базой данных предоставляется библиотека SQLAlchemy в сочетании с Alembic. Первая библиотека - это ORM, которая позволяет конструировать запросы к базе данных в зависимости от выбранного адаптера. Таким образом, сервер может в любой момент начать использовать вместо MySQL PostgreSQL. Alembic поддерживает актуальность текущей базы данных при помощи системы миграций: он автоматически сканирует модели проекта и на основе их содержания собирает набор инструкций, позволяющих актуализировать базу данных до указанных моделей.

Изобилие и разнообразие библиотек невозможно контролировать без пакетного менеджера, который будет следить за совместимостью всех указанных библиотек и версии Python, и для этого можно использовать пакетный менеджер Poetry. Он создаёт собственное окружение в зависимости от указанных библиотек.

Python - гибкий и универсальный язык программирования, с помощью которого можно написать любое программное обеспечение, и позволяющий сконцентрироваться в первую очередь на самых важных решениях.

					ОБЗОР ПРЕДМЕТНОЙ ОБЛАСТИ	Лист
Изм.	Лист	№ докум.	Подп.	Дата		27

1.5.2 СУБД Postgres

PostgreSQL - это современная объектно-реляционная СУБД с открытым исходным кодом. Она является альтернативой коммерческим решениям вроде Oracle Database, а также предоставляет широкий набор функционала, недоступный в других базах данных. На сегодняшний день PostgreSQL, согласно опросам разработчиков, является самым популярным выбором среди баз данных.

Postgres покрывает большинство потребностей, которые могут возникнуть у разработчиков благодаря своему набору функций. База данных поддерживает широкий список возможных индексаций для тонкой настройки, возможность использования JSON в качестве значений колонок а также поиск по этим значениям, высокая эффективность и утилизация подключений благодаря Multiversion concurrency control.

И даже если текущего функционала в Postgres недостаточно, всегда можно расширить его, сделав инструмент максимально подходящим для решения поставленных задач.

Самое главное - инструмент достаточно взрослый и для него существует очень много адаптеров и библиотек для работы с ним. Есть возможность настроить миграции и без проблем использовать Postgres в качестве базы данных для ORM благодаря высокой поддержке языка запросов SQL.

1.5.3 Nginx

Nginx - современный открытый веб-сервер, пришедший на смену коммерческому решению в виде Apache Server.

Главным преимуществом Nginx является то, что он умеет способен работать в многопоточном режиме. Обычная настройка Nginx уже довольно неплохо справляется с множественным обращением пользователей, но её можно изменить увеличив количество исполнителей и количество соединений, которое может обрабатывать исполнитель. Количество открытых соединений связано с ограничениями операционной системы и ограничено количеством открытых файлов, а количество исполнителей нет смысла ставить больше чем количества ядер процессора.

Ещё одной особенностью Nginx является продвинутый роутинг запросов при помощи настройки location в конфигурационном файле. При помощи продвинутого фильтра можно настроить выдачу файлов как с бекенда, так и отдавать данные необходимые для отображения фронтенда.

Nginx может выступать и как Прoxy сервер, перенаправляя запросы на внутренне открытые ресурсы если они доступны и наоборот отклонять запросы если

					ОБЗОР ПРЕДМЕТНОЙ ОБЛАСТИ	Лист
Изм.	Лист	№ докум.	Подп.	Дата		28

они ведут на защищённый ресурс.

Настройка сервера остаётся как достаточно лёгкой, так и достаточно гибкой что позволяет решать различные задачи связанные с перенаправлением на нужные ресурсы.

1.5.4 JavaScript

JavaScript - это скриптовый язык программирования предназначенный для запуска внутри браузеров. Язык программирования строится на основе стандарта ECMAScript, разрабатываемый организацией ECMA. На основе языка программирования ECMAScript существуют приложения написанные для Node.JS, например веб-серверы, использующие Express.

JavaScript имеет специфичные для браузера API, вроде объекта window, позволяющий получить доступ к DOM страницы, navigator содержащий информацию о браузере. Кроме того, JavaScript в браузерах всегда однопоточный. Да, в JavaScript есть генераторы и асинхронные функции, но они не являются по-настоящему отдельным потоком. Операции чтения и вызовы yield, await приостанавливают поток выполнения внутри функции и движок переключается на другую функцию.

Одной из особенностей JavaScript является то, что у него динамическая типизация. И это может быть как удобно, так и неудобно. Динамическая типизация позволяет написать код очень быстро, но поддерживать его становится очень сложно из-за того что контракты между модулями неявные.

Эту проблему решает расширение JavaScript - TypeScript, вводящий статическую типизацию и проверку типов на уровне компиляции. TypeScript, разработанный Microsoft и выпущенный в 2012 году, является высокоуровневым языком программирования со статической типизацией, построенный на основе JavaScript. TypeScript предоставляет возможность описывать свои интерфейсы, типы и классы а также на этапе компиляции проверяет их согласованность. Введение статической типизации позволяет писать более сложные программы.

Одним из наиболее популярных подходов на сегодняшний день при организации графического интерфейса программы является реактивное программирование. Это крайне удобный подход, позволяющий подписываться на изменения состояния и перерисовывать компоненты в зависимости от их изменения. Такой библиотекой для JavaScript является React.

В основе React лежит движок Fiber, которая отслеживает изменения состояния и перевыполняет функции отрисовки и просчётов внутри компонентов, лежащих в дереве. React после своего выхода начал быстро набирать свою популярность благо-

					ОБЗОР ПРЕДМЕТНОЙ ОБЛАСТИ	Лист
Изм.	Лист	№ докум.	Подп.	Дата		29

даря такому подходу. React решал несколько проблем: теперь зависимости внутри компонентов строго иерархические и переходили снизу-наверх, что увеличивало поддерживаемость кода; иерархичность а также автоматически отслеживаемое состояние позволяло быть компонентам всегда в самом актуальном состоянии; очень большая скорость отрисовки и перерисовки компонентов. Конечно, библиотеку можно сломать ненужными вызовами, обилием `useEffect` и применением прочих антипаттернов. Однако, это всё ещё довольно сложно, именно поэтому на просторах интернета всё ещё встречаются плохо написанные приложения React которые однако несмотря на плохую оптимизацию, продолжают работать достаточно быстро.

Существуют и другие библиотеки, использующие реактивный подход в других целях. Библиотека React может использоваться для отрисовки компонентов, однако бизнес-логику стоит делать независимой от более низкого компонента в виде отрисовки. Для этих целей может использоваться библиотека `RxJS`, предоставляющая семейство функций для оповещений и подписки на оповещения. Чаще всего эта библиотека используется в другом фреймворке для отрисовки - Angular. Но ввиду её большой популярности, простоты использования и применения подходов функционального программирования, её можно адаптировать и в React приложении при помощи специальных функций хуков, выносящих состояние из `RxJS` в состояние React. `RxJS` предоставляет большое количество утилитных вспомогательных функций мультикаста, сохранения изначального значения, подписки на состояния DOM-элементов, работу с HTTP и различных преобразований в функциональном стиле, что позволяет хранить бизнес логику чистой, ясной и понятной.

Для дополнительной обработки можно использовать библиотеку `ts-pattern`, которая позволяет проверить содержимое объекта по заданному паттерну.

Как уже было сказано ранее, API валидирует приходящие в неё данные. Часто хорошим тоном является перенос большинства правил на фронтенд, чтобы избежать отправки данных на сервер и не заставлять пользователя ждать. Существует множество библиотек, позволяющие задать парсинг для объектов. В данной ВКР можно использовать библиотеку `Yup` в связке с `react-hook-forms`, предоставляющую широкие возможности для задачи правил и сохранения состояния формы.

Процесс сборки обеспечивает библиотека `Vite` - относительно молодой сборщик, умеющий работать со многими фреймворками. Основным его отличием от устаревшего `Webpack` является гораздо большая скорость сборки. Но для `Vite` всё ещё достаточно мало вспомогательных модулей. Для данного проекта, в остальном, его будет достаточно.

					ОБЗОР ПРЕДМЕТНОЙ ОБЛАСТИ	Лист
Изм.	Лист	№ докум.	Подп.	Дата		30

На смену традиционному пакетному менеджер rpm приходит rpm. Это альтернативный пакетный менеджер, основная цель которого - альтернативный механизм построения дерева зависимостей, направленный на переиспользование ранее загруженных пакетов при помощи ссылок. rpm же обычно пытается загрузить все библиотеки, что влияет на его производительность а также на занимаемое место.

1.5.5 Инструмент для контейнеризации Docker

Docker - это набор программ, который используя средства виртуализации операционной системы, позволяет создать рабочее пространство для выполнения определённой программы внутри легковесных контейнеров.

Многие называют Docker полноценной виртуальной машиной, хотя это не совсем правда. Виртуальная машина создаёт с нуля всю операционную систему, запускает её ядро и затем транслирует команды поступающие из ядра в операционную систему. В итоге мы получаем изолированную систему, которая ничего не знает о родительской операционной системе. Docker же поступает немного иначе, он тоже инкапсулирует рабочее пространство, однако он это делает внутри процесса, который не создаёт отдельное ядро а напрямую общается с родительским ядром. Именно поэтому Docker контейнеры и сильно легковесней запуска полноценной виртуальной машины: это всё ещё нативные программы.

В репозитории Docker можно найти множество готовых образов для развёртывания. Контейнеризация происходит при помощи файла-рецепта (обычно он имеет название .Dockerfile), который описывает, как подготовить контейнер к работе. В него могут входить команды копирования, запуска Bash или Powershell скриптов, указания базовых контейнеров, на основе которых он и будет создавать новый контейнер. Сам контейнер подготавливается, каждый его шаг кешируется и в итоге мы получаем готовый к работе контейнер. Контейнер можно запустить, передав в него дополнительные настройки: подключенные дисковые пространства, проброс портов из дочернего контейнера в родительскую операционную систему, указание в пределах какой сети он может работать. Таким образом, Docker - инструмент, позволяющий инкапсулировать в себе рабочее пространство и обеспечить его кросс-выполняемость на разных машинах (разумеется, на которых можно установить Docker).

Более высокоуровневым компонентом, позволяющим собирать несколько Docker контейнеров и запускать их вместе, является Docker Compose. Файл конфигурации Docker Compose позволяет установить иерархию зависимостей между контейнерами: указать, какой контейнер зависит друг от друга. Например, бекенд

					ОБЗОР ПРЕДМЕТНОЙ ОБЛАСТИ	Лист 31
Изм.	Лист	№ докум.	Подп.	Дата		

не может запустить без базы данных. А также указать для докер образов дисковые пространства, сети и прочее.

1.5.6 NVIDIA Container Toolkit

Так как контейнеризированное приложение будет использовать аппаратное ускорение в виде ядер CUDA для кодека, необходимо предоставить Docker контейнер в специальном Runtime, который позволит получить доступ к аппаратным средствам NVIDIA.

NVIDIA Container Toolkit - это набор библиотек и утилит, который позволяет собирать и запускать контейнеры, которым нужна аппаратная поддержка графической карты. После установки будет предоставлен специальный runtime для Docker контейнеров, в котором будет возможность подключения к графической карте и использовать её ядра CUDA для вычислений.

					ОБЗОР ПРЕДМЕТНОЙ ОБЛАСТИ	Лист
Изм.	Лист	№ докум.	Подп.	Дата		32

2 Разработка архитектуры программного обеспечения

При разработке архитектуры программного обеспечения необходимо придерживаться определённых концепций, обеспечивающих расширяемость и поддерживаемость продукта. Текущий набор функциональных требований может сильно дополняться и изменяться в будущем, следовательно необходимо в архитектуре приложений оставить возможность их расширения и замены модулей.

Одним из наиболее часто используемых приёмов при построении архитектуры будет декомпозиция задачи на однородные атомарные подзадачи, а также иерархия исключающая циклические зависимости.

Такое разделение задачи может занимать сильно больше ресурсов а также потребуется написать сильно больше кода для определения контрактов между модулями программы. Однако такое усложнение позволит снизить когнитивную нагрузку на разработчика при создании новых и изменении старых модулей программы а также обеспечит их взаимозаменяемость.

2.1 Архитектура интерфейса взаимодействия с сервером

Существует множество подходов к организации интерфейса взаимодействия с сервером, которые имеют свою пользу в определённых сценариях. Рассмотрим несколько вариантов а также определим, подходят ли они нам.

2.1.1 SSR

Первый вариант - использование серверного рендера с отправкой форм для модификации данных. При данном способе взаимодействия с клиентом сервер отправляет графический интерфейс, который описывает в себе взаимодействие с пользователем. Отправка запросов, которые будут модифицировать данные на сервере, обычно происходит при помощи multipart запросов с формой стандартными средствами. Дополнительная обработка и отрисовка на пользовательской стороне обычно минимальна.

Существует множество способов реализации данного принципа, например использование Next.js или же PHP. Множество веб серверов предлагают собственные механизмы позволяющие генерировать шаблоны.

Этот подход имеет свои несомненные плюсы. Например, повышенную безопасность и защиту от CSP атак путём предоставления CSP Nonce в сгенерированный

					Разработка архитектуры программного обеспечения			
Изм.	Лист	№ докум.	Подп.	Дата	Разработка программного обеспечения для прогнозирования погоды на основе анализа спутниковых снимков с использованием методов машинного обучения	Лит.	Лист	Листов
Разраб.	Пахомов В.А.			3.06.26				
Руковод.	Осипов О.В.			3.06.26			33	80
Консул.						БГТУ им. В.Г. Шухова, ПБ-223		
Н. контр.	Бондаренко Т.В.			11.06.26				
Зав. каф.	Поляков В.М.			25.06.26				

файл. Кроме того, нагрузка при отрисовке у клиента становится гораздо меньше. Индексация сайтов с SSR будет проводиться сильно лучше при использовании такого подхода. Такие сайты как GitHub, Google, Netflix используют технологию SSR для отрисовки.

Однако есть и минусы у этого подхода. Возрастает нагрузка на сервер, особенно при очень большом количестве пользователей. SSR разумно вводить популярным сайтам с большими мощностями. Кроме того, объявление единственного интерфейса для взаимодействия в виде SSR оставляет мало манёвра для переноса приложения на другие платформы, вроде десктопных приложений, приложений для мобильных устройств или же использование сервиса в пределах других сервисов. Согласно пункту А.4.4 сервер будет обладать недостаточно большой мощностью для реализации серверной отрисовки, следовательно для данного проекта SSR не подходит

2.1.2 gRPC

gRPC - протокол, построенный на основе HTTP, позволяющий обмениваться сериализованными бинарными данными, или же протобафами.

Основная проблема, которую решает gRPC - это скорость передачи данных. Протобаф описывается контрактом - строгим набором полей, описывающих каждое поле. После чего на основе этой информации данные сериализуются в бинарный файл, после чего он отправляется и на обратной стороне используя тот же контракт, при помощи которого был собрана модель, получатель может восстановить полученные данные. Кроме того, протокол gRPC передаёт абстрактные модели, а следовательно не привязан к конкретному способу отрисовки, а значит может быть переиспользован среди разных платформ.

Однако существуют и минусы, основным из которых является поддержка консистентности контракта. В командах, где используется gRPC часто содержится общий репозиторий, который содержит эти контракты. При обновлении сервера и клиента необходимо строго отслеживать совпадение контрактов на сервере и клиенте, что может быть затруднено. Кроме того, инструментов для работы с протобафами на клиенте достаточно мало на сегодняшний день - отладка может быть затруднена.

Протокол gRPC часто используют в межсервисном взаимодействии в микросервисной архитектуре, где множество экземпляров серверов должны постоянно общаться между друг другом. Он отлично справляется с задачей оптимизации запросов а также уходит от задачи отображения и передаёт лишь абстрактные бизнес сущности. Но для текущей задачи он тоже не будет подходить в виду сложности его отладки.

					Разработка архитектуры программного обеспечения	Лист
Изм.	Лист	№ докум.	Подп.	Дата		34

2.1.3 WebSocket + STOMP

WebSocket, в отличие от SSR и gRPC, использует не сессионные передачи данных, а поддерживает постоянный двунаправленный поток передачи данных. Использование STOMP - расширения WebSocket - позволяет ввести авторизацию для пользователей и аутентификацию а также систему подписку на только необходимые каналы, что облегчает канал и позволяет оптимизировать его.

WebSocket часто используется в проектах, в которых важна мгновенная реакция на изменения

Постоянное подключение по WebSocket позволяет построить event based систему, которая будет реагировать на изменения данных на сервере и отправлять их на клиент, что совпадает с выбранным инструментарием в виде React и RxJS на клиентской части - это естественная для сервиса система. Использование вебсокетов позволяет поддерживать данные всегда в актуальном состоянии. Согласно пункту А.4.1.е.3 вебсокет был бы крайне удобен для стриминга, так как он бы смог отправлять данные постепенно и порционно без разрыва соединения по возможности.

Однако у технологии пока что очень слабо развиты инструменты для разработки документации и автогенерации типов. Они есть, вроде документации AsyncAPI. Однако её приходится писать вручную, что не очень удобно. При большом потоке пользователей пропускные каналы могут очень быстро забиться - утилизация бесполезных каналов ничего не отправляющих однако всё ещё держащих связь будет очень низкой. Согласно требованиям технология постоянного поддержания связи могла бы быть полезна только в случае пункта А.4.1.е.3, однако большинство пунктов А.4.1 вебсокет не нужен. В функциональных требованиях не указана необходимость поддержания актуального состояния отображаемого контента. Данный способ общения мог бы подойти для работы с аудиоданными, однако для большей части информации он будет бесполезен.

2.1.4 REST

Большая часть функциональных требований, описанных в техническом задании, не требуют постоянного соединения. Согласно пункту А.4.1.е.3 нам мог бы понадобиться продвинутый кодек для стриминга, вроде WebRTC для стриминга аудиоданных. Однако согласно пункту А.4.1.ж.2 мы также должны иметь возможность выбрать время проигрывания трека, кодеки направленные исключительно на стриминг и мультикастинг здесь бы не подошли.

Для обозначенных задач будет достаточно использования контрактных запро-

					Разработка архитектуры программного обеспечения	Лист
Изм.	Лист	№ докум.	Подп.	Дата		35

сов вроде RESTful API построенным на основе HTTP. Исключением может быть трансляция аудиоданных, однако запросы этого типа могут поддерживать Keep-Alive соединения, не ожидающие полного окончания передачи данных.

Кроме того, функциональные требования описанные в пунктах А.4.1 явно диктуют операции создания, удаления, прочтения, получения списка, обновления. В соответствие этим методам ставятся методы POST, DELETE, GET, GET, UPDATE/PATCH.

Обязательным требованием А.4.1.а должна производиться аутентификация пользователя. Для этого можно использовать либо Cookie либо же JWT токен. Первый метод аутентификации заключается в том, что сервер просит клиент ”запомнить” определённый идентификационный номер и прикреплять его к каждому запросу автоматически. Идентификационная Cookie может содержать подписанный ключ, подтверждающий личность пользователя. С использованием этого подхода, однако, существуют риски CSRF атак. Более современным и часто используемым методом является прикрепление к каждому запросу JWT токена, который также подтверждает личность пользователя, но уже в отличие от Cookie хранится внутри оперативной памяти устройства. С использованием JWT связано гораздо меньше рисков безопасности, его вполне можно использовать для текущего проекта.

Схема создание токена, используемая в сервисе, описана на рисунке 2.1

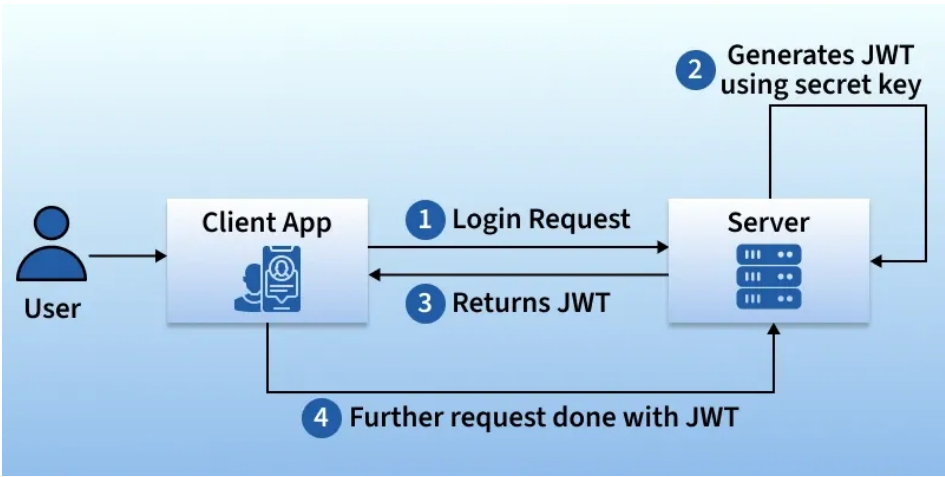


Рисунок 2.1 – Схема создания и использования аутентификационного токена [4]

REST покрывает все требования, его можно использовать в проекте.

2.2 Разработка структуры нейронной сети

При проектировании нейронной сети необходимо учитывать специфику аудиоданных а также функциональные требования.

Спецификой сжимаемых данных являются аудиоданные. Это значит, что они имеют переменную длину. Одним из способов, который позволит работать с данными нефиксированной длины является дополнение данных "мусором". Такой подход сильно облегчает архитектуру нейронной сети, однако производительность такого решения будет сильно падать, так как закодированная последовательность будет во-первых очень большой а во-вторых может потенциально содержать слишком много лишних данных. Однако выбранная архитектура нейросети автоэнкодера не предполагает изменяемый размер данных. С этой задачей мог бы справиться трансформер, однако он изначально решает другой кластер задач.

Решение этой проблемы заключается в разделении непрерывных аудиоданных в последовательность независимых друг от друга кусочков аудио фиксированной длины. Фиксированная длина позволит использовать автоэнкодеры. Мы потеряем возможность получения единой аудиозаписи единственным проходом а также придётся архитектурно усложнять процесс декодирования/кодирования информации. Однако так же благодаря тому что кусочки аудио будут независимы можно будет выполнять процессы декодирования/кодирования в мультипоточном формате. Кроме того, согласно требованию А.4.1.е.3 мы сможем выполнять потоковую распаковку данных. Данное решение позволит реализовать как постепенную потоковую распаковку, так и оптимизированное декодирование с использованием нескольких потоков.

При проектировании нейронной сети необходимо учитывать контекстуальную природу аудиоданных. Как уже было сказано ранее, музыкальные аудиоданные подчиняются определённым ритмическим и гармоническим правилам, которые, как правило, существуют внутри контекста. Темп - это частота сильных приоритетных долей композиции, часто обозначается в качестве ВРМ (удары в минуту). Наиболее распространённым темпом музыкальных композиций является 120 ударов в минуту что будет равно 2 ударам в секунду. В идеале контекстное окно должно содержать 1 секунду воспроизводимого окна для содержания темповой информации.

Не менее важной будет информация и о частотах. Так, например, период самой длинной слышимой человеком волны в 20 Гц будет составлять около 0,05 секунд. Для восстановления частотной информации достаточно будет гораздо более маленького окна.

Средняя частота дискретизации аудиоданных составляет 44100 Гц. То есть, за секунду происходит 44100 измерений. Таким образом для сохранения частотной информации будет достаточно 2205 параметров, в то время как для сохранения

					Разработка архитектуры программного обеспечения	Лист
Изм.	Лист	№ докум.	Подп.	Дата		37

средней информации в плане темпа понадобится 44100 параметров.

При проектировании архитектуры нейросети, однако, стоит также учесть и возможности самой системы. Так, согласно пункту А.4.4.в мы располагаем 16 гигабайтами видеопамати. На сегодняшний день этого объёма может быть достаточно для запуска готовых моделей, однако для обучения такого объёма памяти может не хватить. Стоит также учесть, что при использовании автоэнкодеров важен плавный переход из большей размерности вектора в более маленький, чтобы не допустить потерю данных.

Так как целью нейросети является гибридная модель взаимодействия с FLAC, нейросеть может дополнять алгоритм в тех местах, где FLAC неспособен оптимизировать хранение данных. Средний коэффициент сжатия FLAC составляет 2, так что автоэнкодер может быть нацелен на сжатие в два раза.

Исходя из требований к автоэнкодеру и аппаратных ограничений при обучении и использовании была получена следующая архитектура нейронной сети, представленная на рисунке 2.2

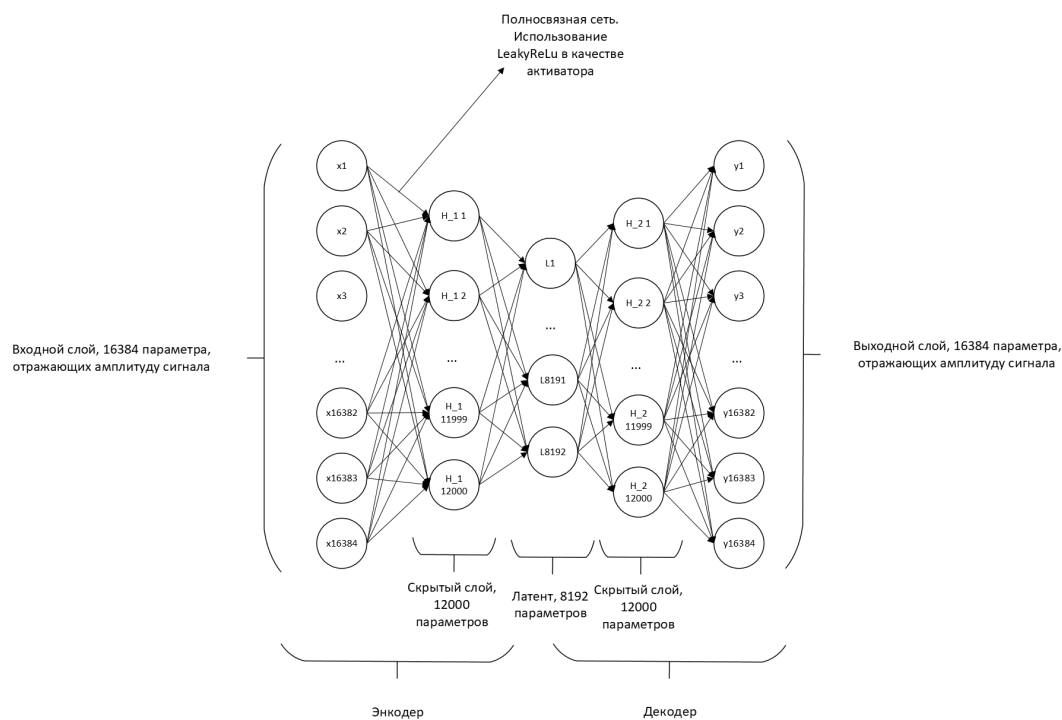


Рисунок 2.2 – Схема автоэнкодера, авторский материал

Полученная нейросеть имеет 589824000 весов. В качестве активаторов используются LeakyReLU, основным их преимуществом является то, что они не отключают нейроны, выдающие отрицательные значения, а всё же дают им внести небольшой вклад.

Данная архитектура имеет свои недостатки: количество входных параметров

ограничено количеством памяти устройства и не может содержать всю информацию о темпе, данный отрезок сможет передать 0.37 секунды аудио, что соответствует примерно восьмой доле; скрытый слой слишком резок в переходе. Однако такую модель легко и быстро обучать, она выполняет кодирование/декодирование достаточно маленьких отрезков аудио. Кроме того, входной слой достаточно большой для сохранения информации о частотах. Ограничения в количестве параметров вызваны аппаратными ограничениями.

В качестве функции потери была выбрана классическая функция `MSELoss`. Как уже было сказано ранее, психоакустические оптимизации могут испортить исходный материал, в данном случае автоэнкодер должен стремиться к восстановлению наиболее близкому к исходному аудио варианту.

Данный автоэнкодер может использоваться в комбинации с кодеком `FLAC` для сохранения данных. Простые нешумные данные могут быть сохранены `FLAC`, а более сложные данные требующие нахождения более комплексных закономерностей могут быть использованы нейронной сетью.

Полученный результат кодирования/декодирования должен быть сохранён в файл. При этом файл должен быть снабжён специальными заголовками для возможности декодирования файлов. Общие данные вроде количества сэмплов, частоты дискретизации можно расположить в начале самого файла. Формат файлового заголовка описан на рисунке 2.3. Эти данные позволят определить длину аудиоданных без надобности декодировать данные полностью.

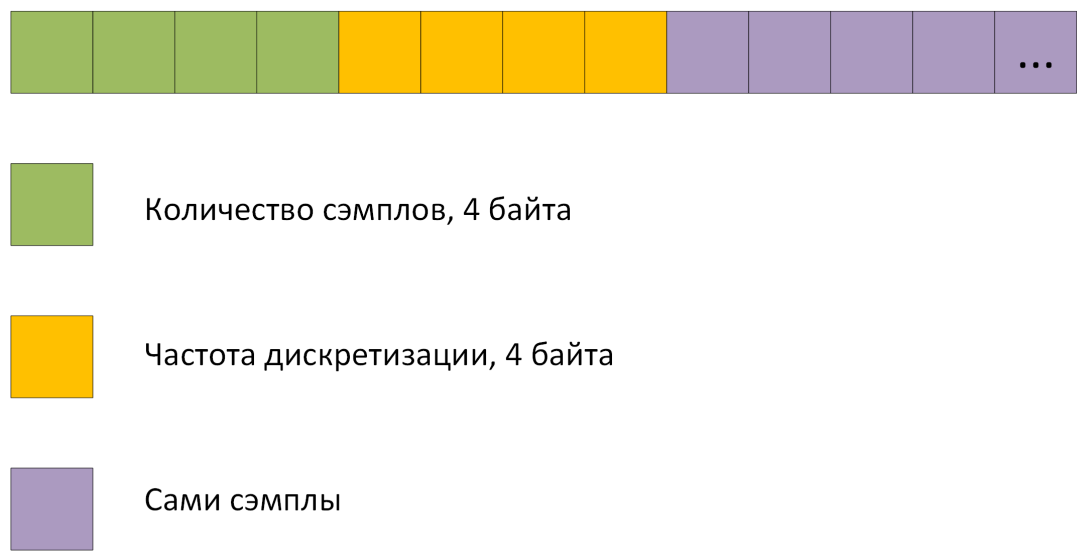


Рисунок 2.3 – Заголовок файла, авторский материал

Так же, так как файл разделён на сэмплы, каждый сэмпл так же описывается своим подзаголовком. Он включает в себя размер и режим кодирования. Режим

кодирования описывает то, с помощью какого формата были записаны следующие данные. Следующие 4 байта описывают длину бинарных данных. После этого идут сами бинарные данные - закодированные при помощи алгоритма Хаффмана или же латент нейросети. Структура заголовка представлена на рисунке 2.4.



Рисунок 2.4 – Заголовок сэмпла, авторский материал

Использование в кодеке привязанных к конкретной нейронной сети данных может вызвать определённые трудности в дальнейшем будущем и совместимости данных. Например, если разработчик захочет обновить нейронную сеть, она скорее всего не сможет декодировать старые данные. Согласно требованиям, однако, кодек не обязан быть распространённым для широкой публики и является специфичным для программы форматом хранения аудиоданных. Поэтому для совместимости нужен специальный инструментарий, преобразующий исходные данные в общепринятый формат.

2.3 Проектирование базы данных

В соответствии с функциональными требованиями А.4.1 необходимо разработать модели базы данных, которые смогут обеспечить хранение всех необходимых данных необходимых для заданной бизнес логики.

При проектировании базы данных будем пользоваться принципами нормализации базы данных, что позволит избежать дублирования базы данных. Целевой степенью нормализации будет являться 3 нормальная форма. При такой степени нормализации обеспечивается достаточное дедублирование данных, но таблица всё ещё достаточно оптимизирована для использования а также легка для анализа разработчиком. Обеспечение связности данных в реляционных базах данных выполняется

при помощи механизмов создания отношений, обеспечивающих автоматическую поддержку консистентности базы данных при помощи условий запрета/обновления связанных полей.

Рассмотрим более подробно структуру базы данных, представленной на рисунке 2.5.

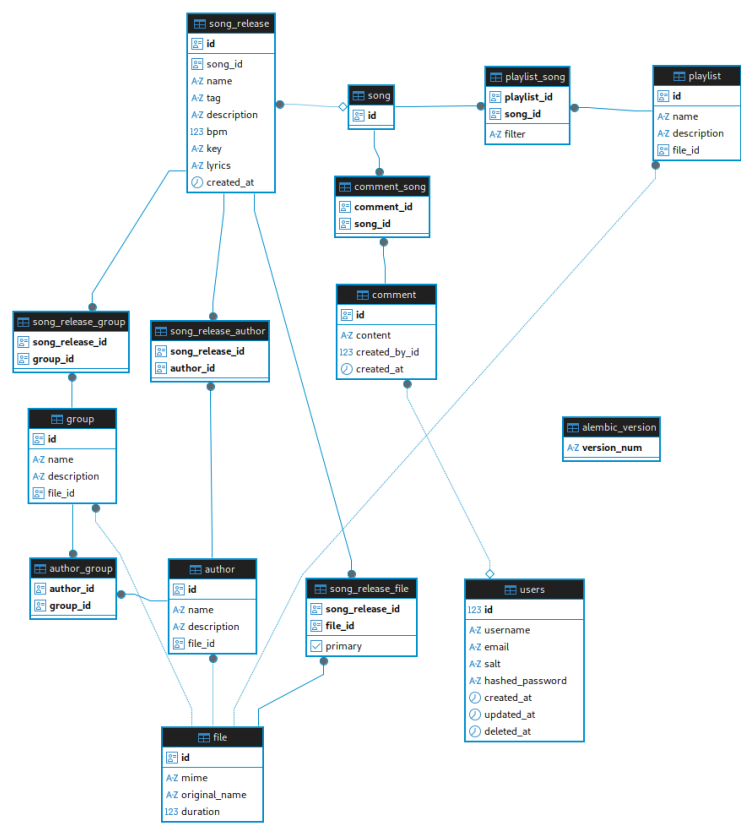


Рисунок 2.5 – Структура базы данных, авторский материал

Одной из наиболее часто встречающихся задач, как и в большинстве сервисов, является хранения данных. Записи о файлах содержатся в таблице file, где так же предоставлен mime тип и оригинальное имя файла. Почему не положить файл внутрь таблицы? Ведь хранение файлов в базе данных напрямую сильно облегчает работу с ними. Можно просто взять файл из запроса. С этим решением связаны механизмы как базы данных, так и библиотек веб серверов. Базы данных не предназначены для хранения больших данных, она оптимизирована для многопользовательской работы и сохранении целостности базы данных. Файлы действительно можно положить в базу данных, но это очень сильно повлияет на скорость работы, особенно при запросах на базовые бизнес модели, содержащие в основном короткие текстовые, числовые данные. Стоит декомпозировать задачу хранения бинарных файлов. Кроме того, фреймворки веб серверов при отправке файлов оптимизированы для работы

именно с классическими файлами, хранящимися на дисковом пространстве. Условия диктуют необходимость переноса задачи хранения файла в дисковое пространство. В модели так же будет предоставлять опциональное поле duration обозначающее длительность аудиоданных. Так как подавляющее большинство файлов на сервисе будут аудиоданными, эта оптимизация имеет смысл. Введение схемы file удовлетворяет требованиям технического задания А.4.1.е а также позволяет хранить изображения в других функциональных пунктах А.4.1.

Таблица users удовлетворяет функциональному требованию А.4.1.а, предоставляя возможность хранения пользователей. В базе данных можно хранить пользователей и идентифицировать их при помощи пары почты и пароля. Важно отметить, что в базе данных хранится hash пароля.

Требования А.4.1.б, А.4.1.в обеспечивается схемами group, author, author_group. Эти схемы содержат необходимые данные а также таблицу связи вида N:N.

Согласно требованию А.4.1.г песня не может быть изменена напрямую. Каждое изменение должно формировать новую версию, поэтому решено было разделить сущность песни и версии песни на две отдельные сущности. Причём, сущность песни, хранящаяся в таблице song, крайне минималистична и содержит только идентификатор. Основные же данные хранятся в версии песни - схеме song_release, содержащей дополнительную информацию. Прочие связи типа N:N обеспечиваются схемами song_release_group, song_release_author, song_release_file. Комментарии хранятся в схемах comment и comment_song. Требование к обеспечению ведущих файлов в А.4.1.г.2 обеспечивается расширенной схемой song_release_file для обеспечения связи N:N путём добавления нового поля primary.

Сущность плейлистов, описанная в требовании А.4.1.д, находится в схеме playlist. Список песен задаётся таблицей playlist_song. Важно отметить, что плейлист ссылается на song, а не song_release. Поиск же по сущностям осуществляется при помощи умного фильтра, описанного в требовании А.4.1.д.1. Это поле находится в song_release. Нестрогая привязка позволяет динамически выбирать конкретные версии песни для плейлиста.

Таким образом, полученная архитектура базы данных позволяет сервису хранить данные описанные в А.4.1. Важно также отметить, что не все данные хранятся в базе данных. Например, А.4.1.ж.2 не обеспечивается при помощи сервера. Перенос данных на клиентское решение является оправданным, так как эти данные являются слишком динамичными. Но в будущем можно расширить текущие базы данных путём сохранения текущего проигрываемого плейлиста, песни и других дан-

					Разработка архитектуры программного обеспечения	Лист
Изм.	Лист	№ докум.	Подп.	Дата		42

ных для возможности восстановления состояния плейлиста на других устройствах пользователя.

2.4 Проектирование серверной части приложения

Ранее мы пришли к тому, что программе необходимо разделение на клиентскую и серверную часть. Это обосновано множеством пунктов вроде оптимизации работы сервера, легкости отладки. Однако разделение на клиент и сервер не позволяет уточнить достаточно глубоко архитектуру сервисной части.

Так или иначе, архитектура кодовой базы сводится к вопросам иерархии и декомпозиции написанного кода. Если код начинает решать слишком много задач а поддерживать его становится сложно, его нужно декомпозировать. Если код начинает зависеть сам от себя вызывая циклические зависимости, следует пересмотреть архитектуру приложения. Даже разделение программы на базу данных, сервер и клиент говорит о том, что одно приложение не может решать все задачи сразу, если оно достаточно большое.

Рассмотрим варианты декомпозиции сервиса с точки зрения запуска копий.

2.4.1 Системная архитектура

Системная архитектура описывает, как сервис разбит на отдельные сервисы и принципы взаимодействия с ними. Приложение уже частично декомпозировано, например задача хранения данных лежит на отдельном сервисе - базе данных. Рассмотрим целесообразность разделения сервиса на отдельные подсервисы.

Микросервисная архитектура предполагает разбиение сервиса на отдельные микросервисы, которые решают исключительно свой набор задач, как на рисунке 2.6.

Такой подход обладает своими преимуществами. Выход из строя одного из микросервисов не остановит работу всего сервиса. Кроме того, микросервисная архитектура гораздо более гибкая и расширяемая. Например, запросы на микросервис А будут достаточно редки, и можно отдать лишь одному серверу запускать лишь единственный экземпляр этого сервиса. Микросервис Б, например микросервис авторизации пользователей, естественно, будет запрашиваться пользователем гораздо чаще, следовательно для обеспечения стабильной работы потребуется большое количество пользователей. Кроме того, работа в экосистеме микросервисов способствует появлению новых сервисов, которые уже будут использовать готовую архитектуру.

Само собой, у такого подхода есть и недочёты. Она кратно дороже. Микросервисная архитектура сильно сложнее в тестировании и обслуживании. Она требует большое количество специалистов, а также требует настройки их взаимодействия

					Разработка архитектуры программного обеспечения	Лист
Изм.	Лист	№ докум.	Подп.	Дата		43

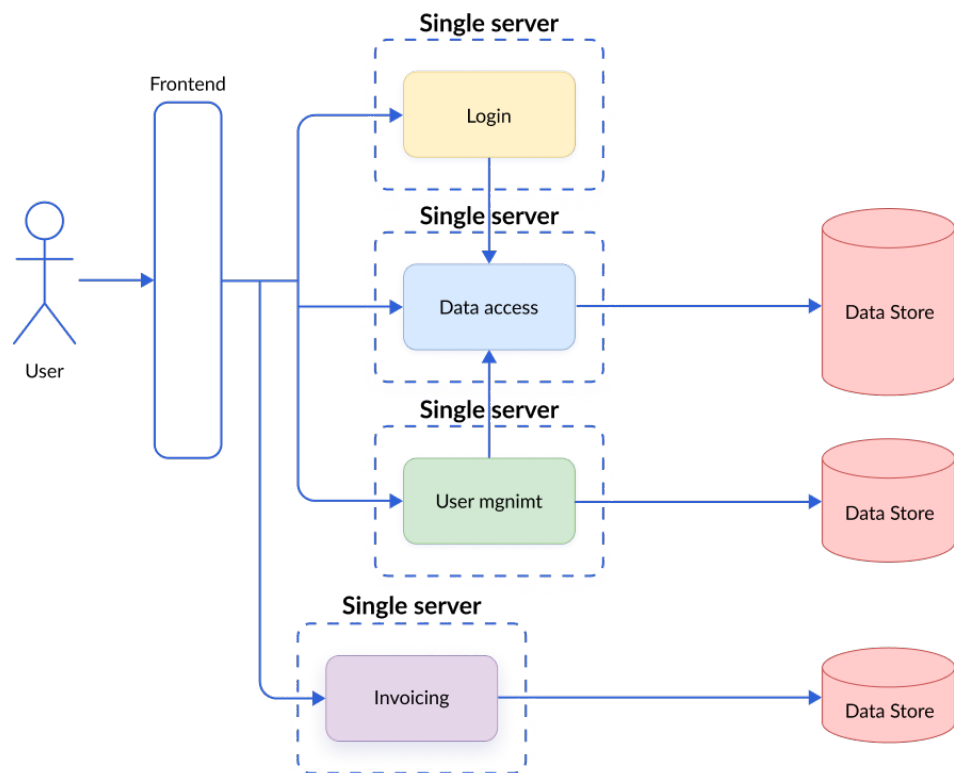


Рисунок 2.6 – Много микросервисов [5]

(этим в команде обычно занимаются DevOps специалисты). Микросервисная архитектура целесообразна исключительно в случаях, когда предвидится крайне большое количество пользователей а также нужна большая переиспользуемость.

Монолитная архитектура предполагает нахождение всей бизнес логики в одном приложении. Чаще такой подход сильно проще для маленьких приложений и для проектирования. Кроме того, накладные расходы на разворачивание инфраструктуры и её поддержку кратно меньше - один сервер купить дешевле, чем много отдельных. Такой подход хорошо подходит для использования небольшим количеством пользователей.

Согласно требованиям А.4.4, А.6 мы не располагаем большим количеством серверов. Кроме того, приложение позиционируется как self-hosted решение, а не готовая сложная архитектура. Целесообразно для данного приложения из экономических соображений выбрать монолитную архитектуру и консолидировать весь код в пределах одного приложения.

2.4.2 Архитектура приложения

Приложение принято разделять на несколько модулей, которые имеют свою задачу как отдельный модуль. Часто этими предназначениями являются хранение данных, обработка этих данных и представление данных. При этом иерархичность слоёв очень важна, и, например, модули, отвечающие за хранение данных, не могут

					Разработка архитектуры программного обеспечения	Лист
Изм.	Лист	№ докум.	Подп.	Дата		44

напрямую вызывать методы отображения. Такая архитектура называется многоуровневой. Рассмотрим архитектуру для сервера на рисунке 2.7.

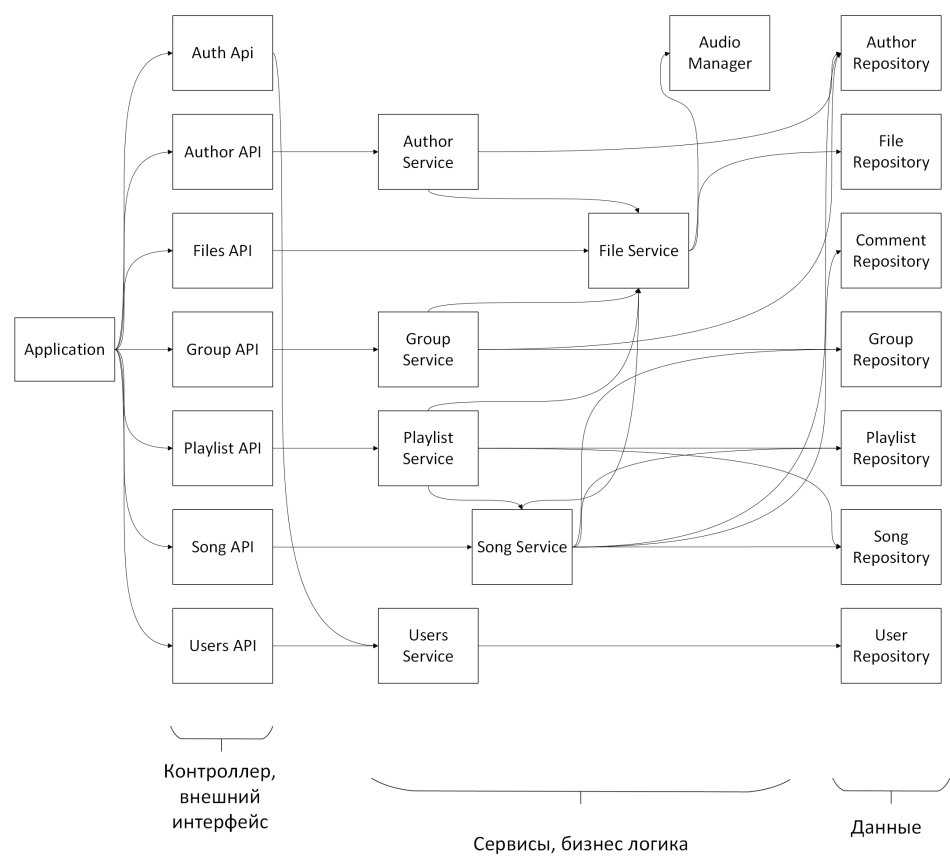


Рисунок 2.7 – Архитектура сервера, авторский материал

Архитектура соблюдает иерархию, циклические зависимости отсутствуют. Каждый модуль предоставляет интерфейс, по которому с ним могут взаимодействовать более низкоуровневые модули. Методы интерфейса определяются тем, что нужно более низкоуровневым слоям приложения. Их также можно назвать юзкейсы.

При этом каждый из модулей может быть реализован в своей собственной парадигме более подходящей для решения текущей задачи. С таким подходом одним из главных инструментов для разработчика будет определение документации при взаимодействии а также экспортируемых модулей. Рассмотрим более подробно архитектуру Audio Manager.

Обзор архитектуры модулей имеет смысл начать делать с более низкоуровневых модулей, от которых уже будут зависеть верхнеуровневые модули. Данный набор пакетов должен решать определённые задачи:

- Он должен быть интегрируемым с другими системами для выполнения задач кодирования/декодирования аудиоданных

- Набор модулей должен иметь возможность быть использованным вне основной программы при помощи более простых инструментов, вроде командной строки.

Иными словами, модули должны иметь различные интерфейсы в зависимости от целей но также должны достаточно хорошо инкапсулировать внутри себя основную логику работы с данными.

Рассмотрим модуль `encode_audio`. Основная задача этого модуля - подготовка внешних данных к обработке собственным алгоритмом. А именно, разбиение аудиоданных на подфреймы и приведение их к бинарному формату амплитудной волны. Этот модуль при выборе альтернативного способа оформления входных данных мог бы разделять аудиофреймы на, например, бинарные изображения с частотой. Архитектура представлена на рисунке [2.8](#).

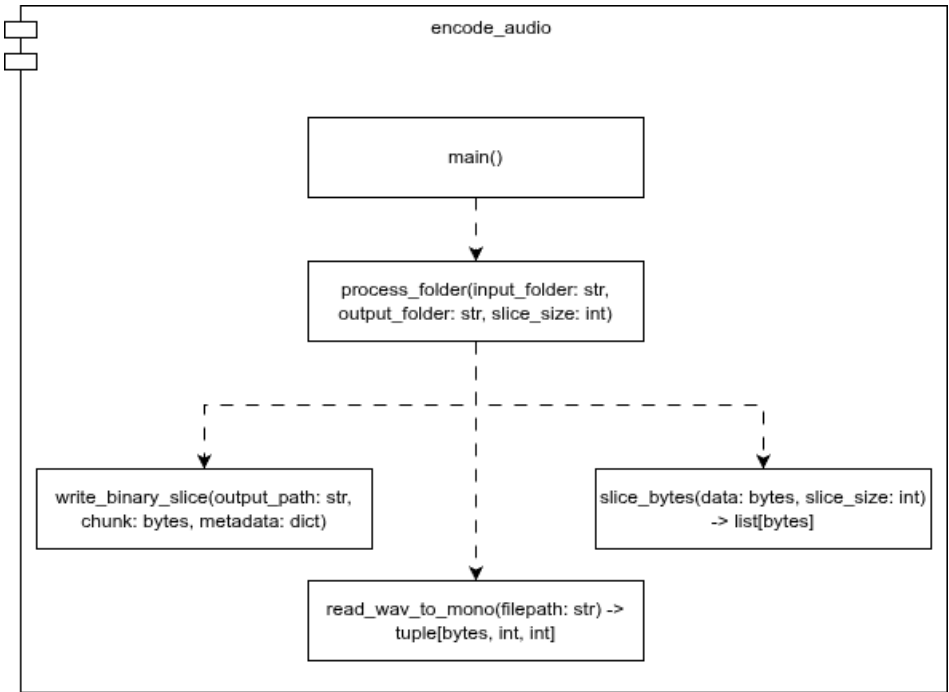


Рисунок 2.8 – Архитектура модуля подготовки аудиоданных, авторский материал

Основная логика инкапсулирована в функции `slice_bytes` а также `read_wav_to_mono`. Первая функция выполняет задачу преобразования аудиоданных во внутренний формат, а `read_wav_to_mono` - нарезает эти данные. Дополнительные функции внутри модуля являются интерфейсом для работы с инструментарием и могут быть адаптированы, например, для консоли.

Перейдём к модулю `decode_audio`. Задача этого модуля обратна задаче модуля `encode_audio`. Здесь мы, наоборот, декодируем бинарные данные в интерпретируемый современными системами аудиоформат. И так же, по возможности, мы можем

изменить формат хранимых бинарных данных. Архитектура этого модуля представлена на рисунке 2.9.

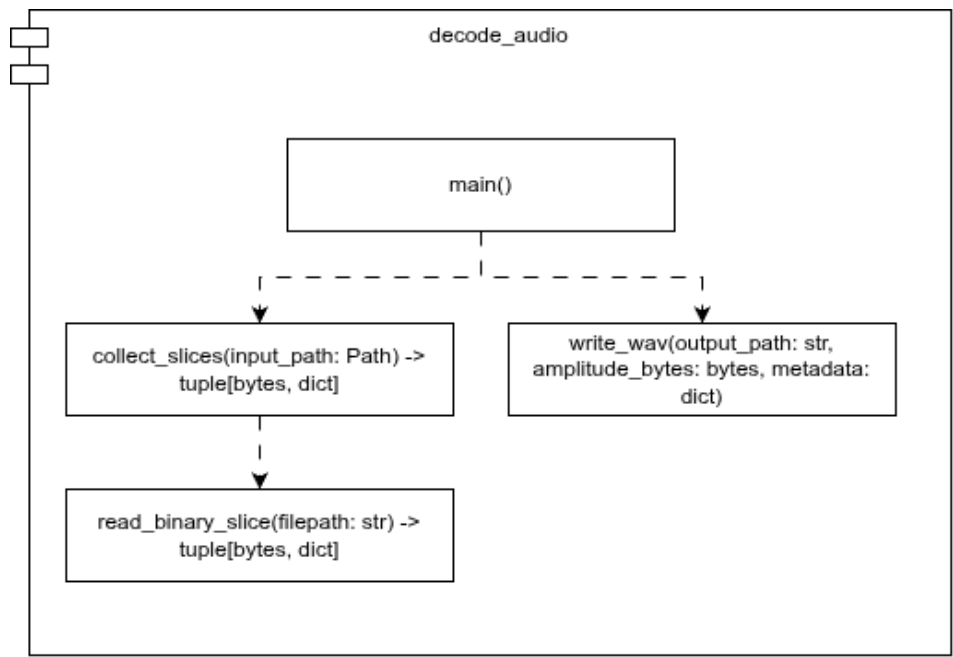


Рисунок 2.9 – Архитектура модуля обратного преобразования аудиоданных, авторский материал

Модуль learner является архитектурно одним из самых сложных. Связано это с тем, что этот модуль инкапсулирует в себе всё что относится к обучению и использованию автоэнкодера. Также метод содержит множество вспомогательных функций, которые решено было инкапсулировать для обеспечения возможности быстрой замены метрик определения качества предсказания. Архитектура этого модуля представлена на рисунке 2.10.

- Все функции внутри модуля можно разделить на несколько категорий
- Общие функции. Например, это функция получения доступного аппаратного ускорителя `get_device`, класс автоэнкодера.
 - Функции и классы направленные на обучение. Класс потери, функции записи и оценки качества предсказания относятся к этой категории.
 - Функции прочтения метаданных. Их относительно мало, но они позволяют получить информацию о том, как обучался автоэнкодер.
 - Функции для предсказания. Это утилитные функции которые позволяют запустить нейросеть, сохранить латент а также восстановить его.

В зависимости от цели эти функции внутри модуля могут быть переиспользованы. Например, инструмент для работы в командной строке для обучения автоэнкодера может использовать функции второй категории. А сервер, который будет только

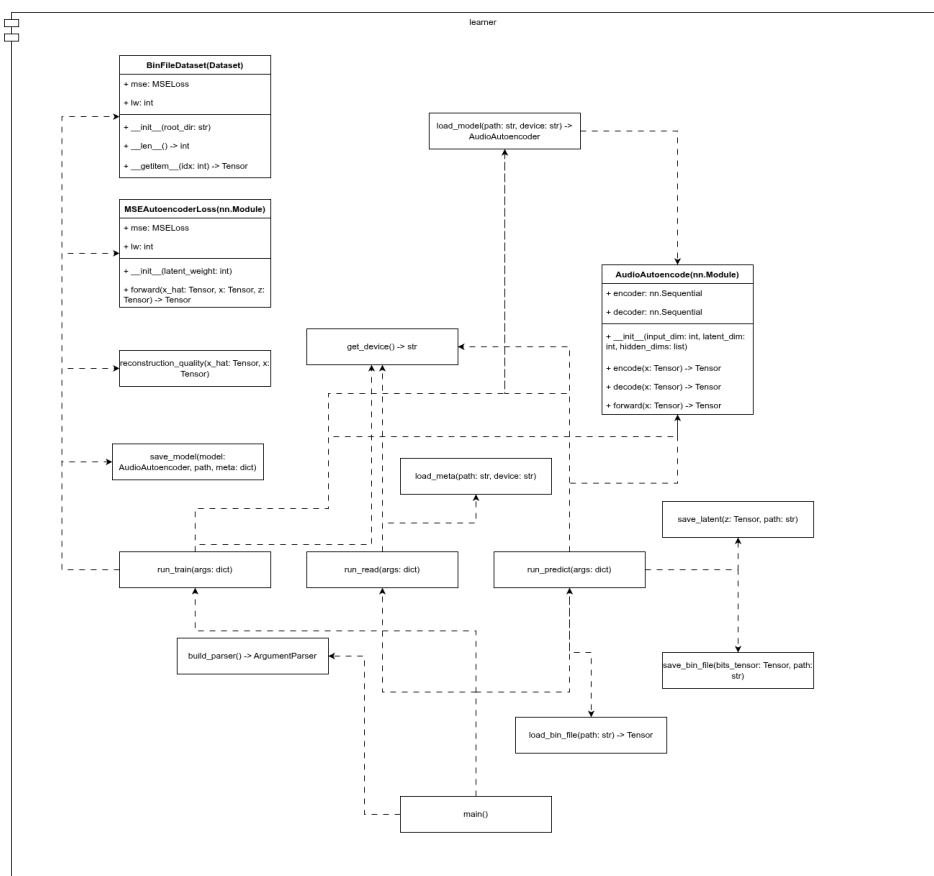


Рисунок 2.10 – Архитектура модуля обучения, авторский материал

непосредственно использовать модель, будет использовать функции 4 категории.

Основной задачей модуля `audio_manager` является упаковка аудиоданных в собственный формат а также потоковая распаковка этих данных. В связи с ограничением возможностей текущих устройств эта распаковка должна происходить в распространённый и известный формат. Для сохранения данных понадобятся модули нарезки а также конвертации данных. Для использования автоэнкодера потребуется модуль `learner` и его класс с автоэнкодером и функцией загрузки автоэнкодера из файла. Межмодульное взаимодействие проиллюстрировано на рисунке 2.11.

При этом сам модуль `audio_manager` отдаёт функции-обёртки для класса `AudioManager`, который организует работу с непосредственным автоэнкодером, что показано на рисунке 2.12

Функции `get_wave_meta`, `get_wav_header`, `decode_stream_to_wave` не включены в базовый класс `AudioManager` так как они выполняют отдельную задачу, а именно преобразование собственного аудиоформата в другой формат. Это функции-адаптеры.

Таким образом, полученная архитектура решает все необходимые задачи описанные в пункте А.4.1.е; удовлетворяет ранее обозначенным требованиям поддерживаемости, иерархичности; даёт возможность адаптации инкапсулированной логики

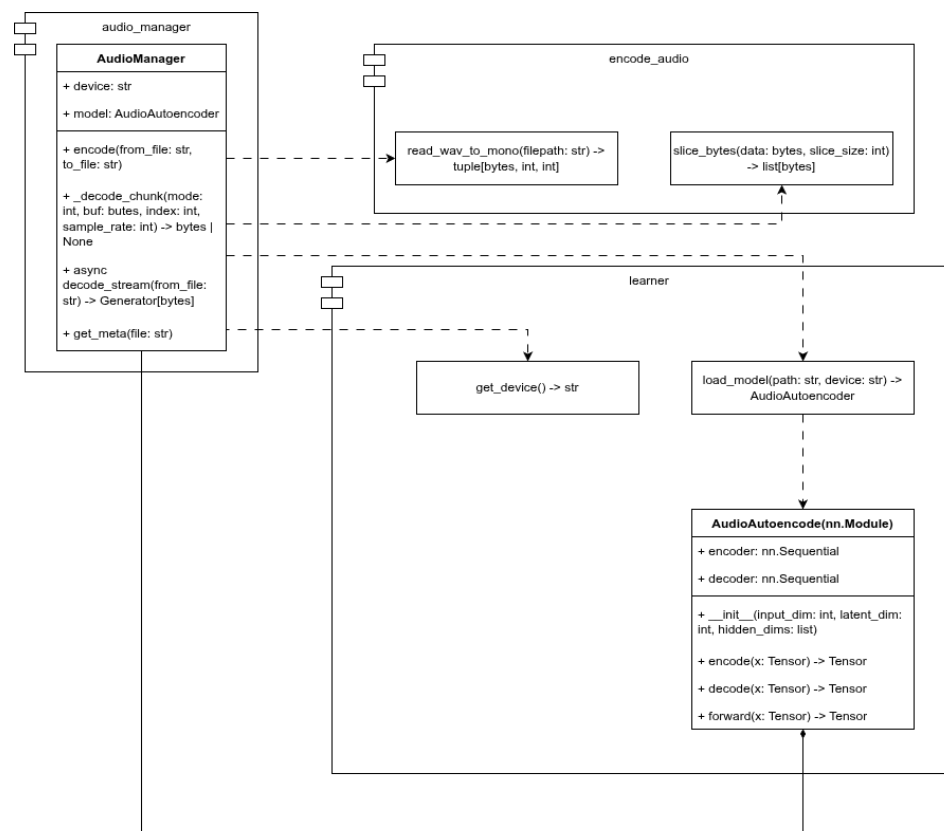


Рисунок 2.11 – Взаимодействие между модулями внутри Audio Manager, авторский материал

в различных сценариях, что в будущем позволит реализовать инструментарий под свои специфичные требования, если таковые появятся.

2.5 Архитектура фронтенд приложения

Как и в других приложениях, в фронтенд приложении существует несколько подходов при организации системной архитектуры приложения. Так или иначе каждая из этих архитектур должна отвечать на запросы управления сложности приложения. Очень большое сложное нагруженное приложение, содержащее весь код в одном файле будет очень сложно поддерживать и модифицировать. А очень простое приложение с десятью ненужными абстракциями будет похоже на игру, за которую никто не будет платить.

На фронтенд приложениях принято использовать крайне упрощённую архитектуру, и в идеале так и должно быть. Если большая часть бизнес логики находится в отдельном приложении, вроде сервиса, то фронтенду ничего обрабатывать не нужно, а его задачи сводятся к отображению запросов. Но часто команды разработки могут быть несогласованы, а ограничения сервиса могут быть вызваны техническими ограничениями. Поэтому часто бизнес логика может оказываться на фронтенде, что будет являться техническим долгом, который, по хорошему, нужно удалять.

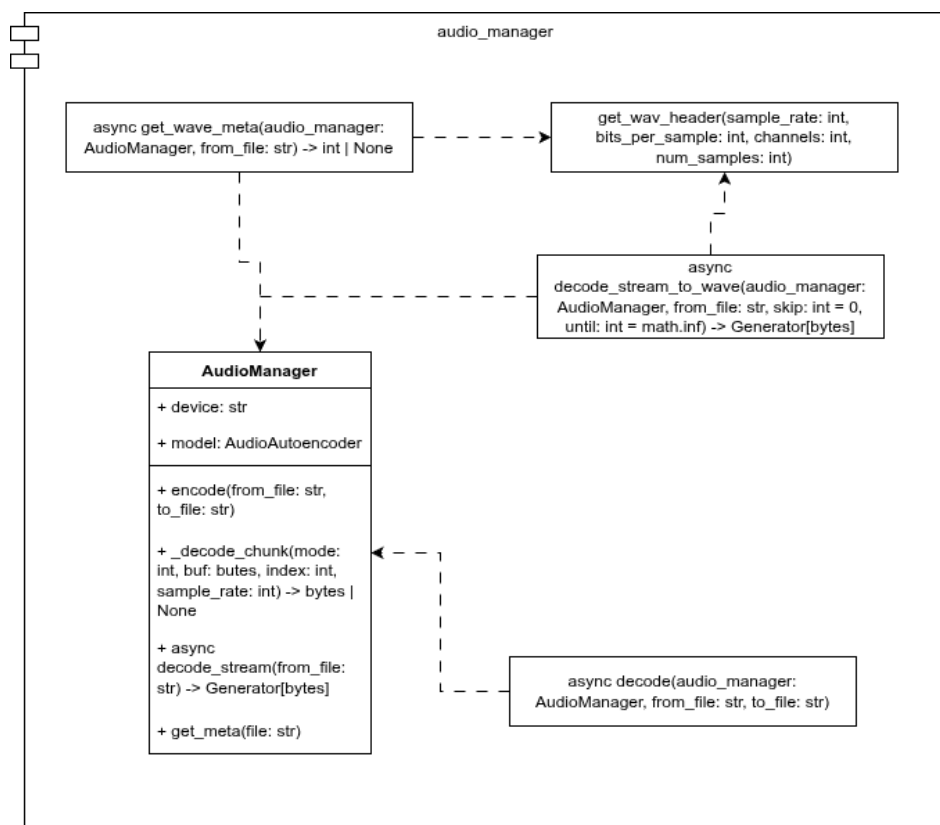


Рисунок 2.12 – Модуль audio_manager, авторский материал

Некоторая сложная обработка данных связанная с визуальным оформлением, сложными формами и превалидацией, работой с внутренними компонентами также могут требовать усложнения на фронтенд приложениях. Отсюда вытекает необходимость в продвинутых паттернах организации кода.

Наиболее популярным архитектурным паттерном в последнее время стал SOLID-ориентированный Feature Sliced Design. FSD вводит слоистую архитектуру и вводит ограничения на возможности использования модулями более высокоуровневых модулей. Каждый слой выполняет свою определённую задачу, однако подход может быть гибким в зависимости от задачи. В то же время слой содержит срез, которые делят слой по домену или же фиче. Каждый слайс же в свою очередь содержит сегменты, которые определяют назначение внутри кода (UI отображение, API, конфигурация и так далее).

Рассмотрим каждый из слоёв внутри приложения а также правила заданные внутри каждого слоя.

2.5.1 Общий слой (Shared)

Общий слой не обладает какой-либо бизнес ценностью и предоставляет общую информацию для всех других слоёв. Этот слой не содержит в себе домены, он содержит только сегменты. Рассмотрим несколько сегментов:

- API может содержать автосгенерированный интерфейс обращения к серверу. Это архитектурное решение обеспечивает типобезопасность ответов и запросов.
- Config содержит конфигурационные переменные, которые задаются при сборке. Обычно, такие переменные содержат путь к серверу. Но могут содержать ещё и настройку фронтенда.
- Lib содержит в себе утилитные переиспользуемые методы
- Model содержит в себе типы, дженерики и прочие данные относящиеся к типам.
- Ui содержит переиспользуемые графические компоненты

Важно отметить, что общий слой должен быть наиболее абстрактными и стабильными. Из первого требование вытекает, как уже было обозначено ранее, отсутствие какой-либо бизнес-логики. А второе требование можно обеспечить различными способами: декомпозиция на маленькие задачи, использование функционального программирования как самой строгой парадигмы не допускающей ошибок.

2.5.2 Слой сущностей (Entity)

Слой сущностей содержит фундаментальную бизнес логику относящейся исключительно относительно к своему домену без учёта других бизнес сущностей. Это "атомы на основе которых и строится всё приложение. Здесь могут находиться правила валидации, способы отображения, enum-значения, схемы домена.

- Model содержит правила полей домена. Длина полей, их содержимое, определение их типа (число, строка или сложный объект).
- Ui содержит представление домена. Это очень легковесные компоненты, так же использующие принципы функционального программирования и идемпотентности при отображении. Такой подход обеспечивает, в том числе, и лёгкость тестирования компонентов.

Этот слой является уже менее абстрактным, так как он определяет конкретные бизнес сущности, которые могут ещё и, пусть довольно редко, но меняться.

2.5.3 Слой фичей (Features)

Если слой сущностей отвечал на вопрос "Из чего сделано приложение? то слой фичей отвечает на вопрос "Что с этими сущностями можно делать?". Слой фичей можно спроектировать по-разному, однако, самое ценное в приложении - всё что оно умеет делать, всё что оно умеет обрабатывать - необходимо тщательно защищать и делать максимально модульным, независимым от фреймворка отображения.

					Разработка архитектуры программного обеспечения	Лист
Изм.	Лист	№ докум.	Подп.	Дата		
						51

В нём необходимо использовать легковесные библиотеки для обеспечения максимальной совместимости между интерфейсами отображения. Этот подход, например, продвигает фреймворк Angular, за что его и очень любят разработчики серверных приложений, так как обычно сервер имеет гораздо более строгие архитектурные правила.

Важно также отметить и способ подключения этой бизнес логики. Один из способов заключается в использовании паттерна Dependency Injection. Согласно этому паттерну наиболее высокий слой инициализирует объекты, а затем внедряет их в компоненты, которые используют его.

- Lib содержит библиотеки, которые могут использоваться в пределах слоя.
- Model - самый сложный и интересный сегмент. Этот сегмент содержит в себе интерфейсы а также реализации этих интерфейсов, содержащие бизнес логику приложения. Интерфейсы обязательно должны определять интерфейсы возвращаемых и принимаемых параметров самостоятельно. Исключением являются базовые типы из слоя сущностей, вроде enum значений. Конвертация значений внутри компонентов допустима, но она должна быть крайне незначительной.
- Service - содержит только логику приложения. Всё что нужно преобразовать, конвертировать, провалидировать. Сервис не может содержать состояние, он должен решать исключительно задачи выполнения действий. Методы интерфейса определяются юзкейсами - тем что нужно сделать на странице пользователю.
- Repository - его задача заключается исключительно в хранении данных. Он так же определяет интерфейс, а реализация может определить способ хранения самостоятельно. Будь то хранение в оперативной памяти или же в постоянной памяти. Преобразований должно быть минимально, преобразования могут быть допущены только для поддержания консистентности хранилища.
- API - этот класс не обязателен, его задача - выполнение непосредственных запросов на сервер. Выделение в отдельный класс может быть оправдано необходимостью создания Mock данных, пока сервер не готов. Такая декомпозиция позволит интегрироваться с сервером при его появлении без проблем.
- Ui содержит простые обёртки относящиеся исключительно относительно фичей которые слишком маленькие для выделения в виджеты. А также этот

					Разработка архитектуры программного обеспечения	Лист
Изм.	Лист	№ докум.	Подп.	Дата		52

слой содержит средства подключения при помощи Dependency Injection.

Заданные правила в пределах этого слоя позволяют сделать этот слой наиболее защищённым от внешних изменений а также наиболее гибким.

2.5.4 Слой виджетов (Widgets)

Виджеты представляют собой мини-приложения. Они содержат всё состояние внутри себя, задают верстку абстрактных компонентов внутри себя а также подключают фича слой для взаимодействия с сервером.

- Ui это единственный сегмент внутри виджетов. Они часто достаточно простые, так как вся сложная логика отдана реализации в сервисах.

2.5.5 Слой страниц (Pages)

Единственная задача слоя страниц заключается в установке верстки страницы и организации виджетов внутри страницы.

- Ui содержит сами страницы. Один компонент - это одна страница.

2.5.6 Слой приложения (App)

Это самый нестабильный и наименее абстрактный слой, задающий реализации. Здесь, как и в общем слое, нет доменов. Здесь только сегменты. Рассмотрим эти сегменты

- Entrypoint - то с чего начинается приложение. Выполняет подключение роутера, контекстов для обеспечения реализации паттерна подключения зависимостей.
- Providers - подключает фичи при помощи контекстов и других библиотек.
- Styles - базовые стили для приложения. Например, CSS-переменные.
- Router - задаёт пути в приложении и подключает страницы.

2.5.7 Архитектура приложения

В соответствии с заданными архитектурными правилами а также функциональными требованиями А.4.1 можем выполнить проектирование фронтенд приложения. Архитектура приложения представлена на рисунке [2.13](#).

Внутри приложения допустимы зависимости внутри слоев, если не нарушается иерархичность. Например, с виджетом player или же с сервисом аутентификации, который нужен всем другим сервисам для аутентификации. Допустимо также отступление от вышеописанных правил. Архитектурный паттерн - это инструмент регуляции сложности, и если из-за него будет слишком большая сложность, он перестанет иметь смысл. Свёртка нижележащих слоёв, например, объединение в

					Разработка архитектуры программного обеспечения	Лист
Изм.	Лист	№ докум.	Подп.	Дата		53

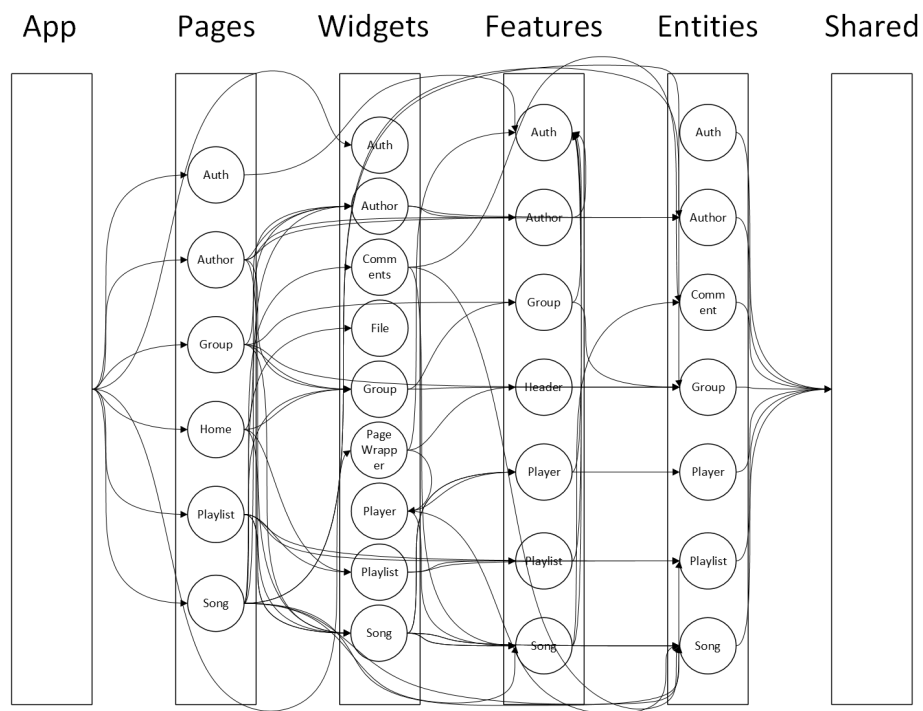


Рисунок 2.13 – Архитектура фронтенд приложения, авторский материал

слое страницы задач виджетов, будет иметь смысл только при переиспользовании виджетов или большой сложности проекта.

Таким образом, полученная архитектура приложения удовлетворяет функциональным требованиям, описанным в пункте А.4.1.

3 Разработка алгоритмов и программного обеспечения

Продиктованные ранее архитектурные решения а также выбор определённого стека технологий позволяет выстроить определённую структуру сервиса.

Сервер использует в качестве хранилища базу данных Postgres, поддерживая её актуальность при помощи средств Alembic а также SQLAlchemy для создания запросов и работы с непосредственными объектами. Эту задачу выполняет самый низкоуровневый компонент - репозиторий.

Сервис содержит бизнес логику, которая обрабатывает входные данные и выполняет вызовы к репозиториям, другим сервисам или системным модулям.

Модули, отвечающие за объявление интерфейса общения с сервисом, обеспечиваются при помощи библиотеки FastAPI. Эта же библиотека позволяет использовать паттерн инъекции зависимостей в виде репозиториев, сервисов и прочего.

Фронтенд использует для отрисовки React, в то время как его бизнес логика содержится в относительно простых классах. Механизм инъекции зависимостей обеспечивается механизмом контекстов.

Для передачи запросов на сервер и клиент используется Nginx, который задаёт пути для отображения документации сервера или же для получения статических файлов сайта. Задаётся также reverse-проxy Nginx сервер, который будет проксировать запросы на экземпляры фронтенда или же сервера.

Для контейнеризации всех приложений используется Docker, а для создания инфраструктуры вокруг этих образов используется Docker Compose.

Рассмотрим каждый из компонентов более детально

3.1 Разработка сервера

Рассмотрим устройство сервера на примере пользовательских сценариев с композициями

3.1.1 Слой хранения данных

Модель, которая будет непосредственно храниться в репозитории, описывается обычным классом и полями из пакета sqlalchemy. Каждая модель наследуется от базовой модели RWModel, представленной в листинге 3.1, которая автоматически свойство __tablename__. Это свойство позволяет определить, как будет называться схема внутри базы данных.

					Разработка алгоритмов и программного обеспечения			
Изм.	Лист	№ докум.	Подп.	Дата	Разработка программного обеспечения для прогнозирования погоды на основе анализа спутниковых снимков с использованием методов машинного обучения	Лит.	Лист	Листов
Разраб.		Пахомов В.А.		3.06.26				
Руковод.		Осипов О.В.		3.06.26			55	80
Консул.						БГТУ им. В.Г. Шухова, ПБ-223		
Н. контр.		Бондаренко Т.В.		11.06.26				
Зав. каф.		Поляков В.М.		25.06.26				

Листинг 3.1 – Базовый класс для моделей базы данных

```

1 from sqlalchemy.orm import declarative_base, declared_attr
2
3
4 class RWModel:
5     @declared_attr
6     def __tablename__(cls) -> str:
7         return cls.__name__.lower()
8
9     __name__: str
10
11
12 RWModel = declarative_base(cls=RWModel)

```

Сама модель уже будет содержать непосредственно как поля, так и средства специфичные для ORM. Рассмотрим более подробно, как описывается модель в листинге 3.2.

Листинг 3.2 – Базовый класс для моделей базы данных

```

1 class Song(RWModel):
2     id: Mapped[UUID] = mapped_column(primary_key=True, default=uuid.uuid4)
3     song_releases: Mapped[list["SongRelease"]] = relationship(back_populates="song",
4     ↪ lazy="selectin")
5     comments: Mapped[list["Comment"]] = relationship(secondary=comment_song, lazy="
6     ↪ selectin")
7     playlists: Mapped[list["PlaylistSong"]] = relationship(back_populates="song", lazy="
8     ↪ selectin")

```

Обычные поля могут быть заданы явно при помощи `mapped_column`, так и только при помощи дженерика `Mapped`. В случае поля `id`, например, мы дополнительно указываем функцию-генератор а также указываем тот факт, что это идентифицирующее поле при помощи `primary_key`. Существуют и другие способы задания типа поля, вроде объекта `Field`, однако этот способ является устаревшим. Важно отметить, что с точки зрения языка мы задаём не сколько поля объекта, сколько статические значения внутри класса `Song`. Потом уже сам `SQLAlchemy` на основе этого шаблона строит сам объект.

Отношения в `SQLAlchemy` реализуются различными способами. Наиболее современный способ - использование `relationship`. Он позволяет установить способ дозапроса объектов при помощи `lazy`, а также указать способ обратного обращения в объекте, на который он ссылается.

Например, модель песни позволяет дозапросить её релизы, а это значит, что в модели `SongRelease` должно появиться свойство `song`, которое задаётся `back_populates`. И так же `SongRelease` должна задать обратную связь, объявив поле `song` и ука-

					Разработка алгоритмов и программного обеспечения	Лист
Изм.	Лист	№ докум.	Подп.	Дата		
						56

зав в `back_populates` значения `song_releases`, что и показано в листинге 3.3. Поле `relationship` таким образом позволяет задавать бинарные отношения между объектами.

Листинг 3.3 – Модель релиза песни

```

1 class SongRelease(RWModel):
2     id: Mapped[UUID] = mapped_column(primary_key=True, default=uuid.uuid4)
3     song_id: Mapped[UUID] = mapped_column(ForeignKey("song.id", ondelete="CASCADE"),
4     ↪ index=True, nullable=True)
5     song: Mapped["Song"] = relationship(back_populates="song_releases", lazy="selectin")
6     created_at = Column(DateTime, server_default=text("now()"), index=True)
7     name: Mapped[str]
8     tag: Mapped[str]
9     description: Mapped[str]
10    bpm: Mapped[int]
11    key: Mapped[str]
12    lyrics: Mapped[str]
13    authors: Mapped[list["Author"]] = relationship(secondary=song_release_author, lazy="
14    ↪ selectin")
15    groups: Mapped[list["Group"]] = relationship(secondary=song_release_group, lazy="
16    ↪ selectin")
17    files: Mapped[list["SongReleaseFile"]] = relationship(back_populates="song_release",
18    ↪ lazy="selectin")

```

Данный вид связи описывает связь вида 1 к N, однако на практике часто возникает необходимость создания записей вида N к N. В реляционных базах данных на данный случай существует паттерн, заключающийся в том, что для обеспечения такой связи добавляется ещё одна схема с двумя связями 1 к N к целевым схемам. Такую связь в SQLAlchemy можно создать относительно легко, как показано в листинге 3.4, так и, если необходимо добавить дополнительную информацию относительно самой связи, при помощи создания обычной полноценной схемы, как показано в листинге 3.5. Первая объявляет простую схему, связывающую релизы композиции и авторов.

Листинг 3.4 – Простая модель для обеспечения связи N к N

```

1 song_release_author = Table(
2     "song_release_author",
3     RWModel.metadata,
4     Column("song_release_id", ForeignKey("song_release.id", ondelete="CASCADE"),
5     ↪ primary_key=True),
6     Column("author_id", ForeignKey("author.id", ondelete="CASCADE"), primary_key=True),
7     )

```

Вторая же схема кроме задания связи между релизом песни и файлом так же задаёт и признак основного файла в релизе композиции `primary`.

Листинг 3.5 – Усложнённая модель для обеспечения связи N к N

```

1 class SongReleaseFile(RWModel):
2     song_release_id: Mapped[UUID] = mapped_column(ForeignKey("song_release.id", ondelete
    ↪ "CASCADE"), index=True, primary_key=True)
3     file_id: Mapped[UUID] = mapped_column(ForeignKey("file.id", ondelete="RESTRICT"),
    ↪ index=True, primary_key=True)
4     primary: Mapped[bool]
5
6     song_release: Mapped["SongRelease"] = relationship(back_populates="files", lazy="
    ↪ selectin")
7     file: Mapped["File"] = relationship(back_populates="song_releases", lazy="selectin")

```

Одним из способов сохранения целостности базы данных является задание определённых ограничений, или же Constraint. Эти ограничения обеспечивают целостность базы данных. Первые базы данных очень часто пользовались этими ограничениями. Это могли быть ограничения на дату, на значения или любые другие сложные ограничения. На текущий момент в разработке роль ограничений прямо в базе данных стала сильно меньше, так как они содержат бизнес-логику, которая находится в другом месте в приложении - в сервисе. Однако полностью ограничения не ушли, например отношения задаются как раз при помощи таких ограничений. И есть множество способов разрешения нарушения этих ограничений, которые в SQLAlchemy в поле типа ForeignKey задаются при помощи параметров ondelete, onupdate, которые могут принимать специфичные для базы данных значения вроде CASCADE, SET NULL, RESTRICT, NO ACTION. CASCADE говорит о том, что при удалении или изменении связанной записи связанную запись необходимо удалить, SET NULL - установить в поле значение NULL, RESTRICT полностью запрещает операцию и потребует ручного обеспечения целостности базы данных, а NO ACTION ничего не делает с записью. Например в листинге 3.5 поле file_id - это внешний ключ, и при удалении файла из базы данных с таким идентификатором все записи, связывающие этот файл с релизом, будут также удалены.

Существует несколько причин, почему такая часть бизнес логики не выносится в предназначенный сервис. Как минимум, это удобно. Ручная проверка целостности таблицы заняла бы очень много времени. А валидация полей и значений вручную приносит больше пользы, чем вреда, так как теперь эти правила можно хранить в изменяемом пространстве без необходимости выполнять миграции базы данных. В итоге, так или иначе, можно прийти к выводу, что любой архитектурный паттерн не является "серебрянной пулей решающей все проблемы, и слепое следование паттерну может привести только к усложнению проекта. Задача разработчика - адекватно оценивать положительные и отрицательные стороны своего выбора.

					Разработка алгоритмов и программного обеспечения	Лист
Изм.	Лист	№ докум.	Подп.	Дата		58

Файл `__init__.py` экспортирует все модели, делая их доступными в других файлах, как показано в листинге 3.6.

Листинг 3.6 – Экспорт моделей

```
1 from .author import Author
2 from .comment import Comment
3 from .comment_song import comment_song
4 from .file import File
5 from .group import Group
6 from .group_author import group_author
7 from .playlist import Playlist
8 from .playlist_song import PlaylistSong
9 from .song import Song
10 from .song_release import SongRelease
11 from .song_release_author import song_release_author
12 from .song_release_file import SongReleaseFile
13 from .song_release_group import song_release_group
14 from .user import User
```

Начало работы Alembic начинается с конфигурационного скрипта, описанном в листинге 3.7. Наиболее важным пунктом является `script_location`. Он задаёт, где будет находиться окружение Alembic, в котором будут анализироваться модели, где будут находиться сами миграции и прочее.

Листинг 3.7 – Конфигурация Alembic

```
1 [alembic]
2 script_location = app/database/migrations
3 prepend_sys_path = .
4 version_path_separator = os
5
6 [loggers]
7 keys = root,sqlalchemy,alembic
8
9 [handlers]
10 keys = console
11
12 [formatters]
13 keys = generic
14
15 [logger_root]
16 level = WARN
17 handlers = console
18 qualname =
19
20 ...
```

Само окружение содержит файл `env.py`, который запускает процесс миграции. Большая его часть сгенерирована автоматически, и самым важным моментом

является определение ссылки на базу данных, которая берётся из настроек, а также метаданные для анализа Alembic. Согласно листингу 3.1, класс RWModel снабжается дополнительными полями благодаря вызову declarative_base. Одним из таких значения является значение metadata, сохранённого в target_metadata. Это поле содержит все схемы созданных моделей и позволяет выполнять запросы в виде отношений. И так как все классы наследуются от одного единственного класса RWModel, metadata у наследников этого класса общая. Этот объект Alembic и использует для определения целевого состояния базы данных, как показано в листинге 3.8.

Листинг 3.8 – Конфигурация окружения Alembic

```

1 SETTINGS = get_app_settings()
2 DATABASE_URL = SETTINGS.db_url
3
4 config = context.config
5
6 fileConfig(config.config_file_name)
7
8 target_metadata = RWModel.metadata
9
10 config.set_main_option("sqlalchemy.url", str(DATABASE_URL))

```

В результате получаются автоматически сгенерированные файлы миграции, которые впоследствии запускаются при помощи инструмента в командной строке для апгрейда или же даунгрейда текущей базы данных. Пример такой миграции есть в листинге 3.9, эта миграция создаёт таблицу авторов при апгрейде и удаляет её при откате.

Листинг 3.9 – Пример миграции Alembic

```

1 def upgrade() -> None:
2     # ### commands auto generated by Alembic - please adjust! ###
3     op.create_table(
4         "author",
5         sa.Column("id", sa.Uuid(), nullable=False),
6         sa.Column("name", sa.String(), nullable=False),
7         sa.Column("description", sa.String(), nullable=False),
8         sa.Column("file_id", sa.Uuid(), nullable=False),
9         sa.ForeignKeyConstraint(
10             ["file_id"],
11             ["file.id"],
12         ),
13         sa.PrimaryKeyConstraint("id"),
14     )
15     # ### end Alembic commands ###
16
17 def downgrade() -> None:

```

```

18 # ### commands auto generated by Alembic - please adjust! ###
19 op.drop_table("author")
20 # ### end Alembic commands ###

```

Таким образом, сформировали наиболее абстрактный слой, обеспечивающий хранение бизнес сущностей. Следующим вопросом при организации кода становится вопрос, как обеспечить работу с моделями внутри приложения? Как уже было обозначено ранее, для подключения зависимостей внутри приложения используется паттерн FastAPI инъекции зависимостей. При таком способе необходимые зависимости подключаются в эндпоинт.

Первым этапом в данном приложении является сама инициализация соединения с базой данных. После этого соединение базы данных сохраняется в контексте FastAPI приложения. Этот процесс представлен в листинге 3.10,

Листинг 3.10 – Инициализация соединения с базой данных

```

1 logger = logging.getLogger(__name__)
2
3
4 async def connect_to_db(app: FastAPI, settings: AppSettings) -> None:
5     logger.info("Connecting to database...")
6
7     engine = create_async_engine(url=str(settings.db_url), pool_size=50, max_overflow=0,
    ↪ echo=True, future=True)
8     async_session_factory = sessionmaker(bind=engine, class_=AsyncSession,
    ↪ expire_on_commit=False, autoflush=True)
9     app.state.pool = async_session_factory
10
11     logger.info("Connected to database.")
12
13
14 async def close_db_connection(app: FastAPI) -> None:
15     logger.info("Closing database connection...")
16
17     # app.state.pool.close_all()
18
19     logger.info("Database connection closed.")

```

При самой инициализации приложения FastAPI вызываются эти методы, которые обеспечивают подключение к базе данных. В дальнейшем для получения доступа к подключению базы данных используется простая функция, которая может быть использована в качестве аргумента функции Depends, которая уже и возвращает подключение к базе данных, как показано в листинге 3.12,

Листинг 3.11 – Функции для инъекции зависимостей для работы с базой данных

					Разработка алгоритмов и программного обеспечения	Лист
Изм.	Лист	№ докум.	Подп.	Дата		
						61

```

1 def _get_db_session(request: Request) -> AsyncSession:
2     return request.app.state.pool
3
4
5 async def _get_connection_from_session(
6     pool: AsyncSession = Depends(_get_db_session),
7 ) -> AsyncGenerator[AsyncSession, None]:
8     async with pool() as session:
9         yield session
10
11
12 def get_repository(
13     repo_type: type[BaseRepository],
14 ) -> Callable[[AsyncSession], BaseRepository]:
15     def _get_repo(
16         session: AsyncSession = Depends(_get_connection_from_session),
17     ) -> BaseRepository:
18         return repo_type(session)
19
20     return _get_repo

```

Работы непосредственной работы с базой данных выполняет репозиторий. Для его получения внутри эндпоинтов используется генератор функции `get_repository`, обозначенный так же в листинге 3.12, который и возвращает реализацию репозитория. Базовый же репозиторий принимает в себя только сессию, базовый класс описан в листинге ??

Листинг 3.12 – Базовый репозиторий

```

1 class BaseRepository:
2     """Base Repository for all repositories."""
3
4     def __init__(self, conn: AsyncSession) -> None:
5         self._conn = conn
6
7     @property
8     def connection(self) -> AsyncSession:
9         return self._conn

```

Таким образом, всё готово для объявления своего репозитория и для работы с моделями. Рассмотрим несколько методов такого репозитория, он представлен в листинге 3.13.

Листинг 3.13 – Методы в репозитории песен

```

1 @db_error_handler
2 async def get_song_releases_short(self, *, filter_: SongFilter, pagination:
3     ↳ SongPagination, sort: SongSort, song_id: UUID) -> list[Song]:
4     query = select(SongRelease).filter(SongRelease.song_id == song_id)

```

					Разработка алгоритмов и программного обеспечения	Лист
Изм.	Лист	№ докум.	Подп.	Дата		62


```

4
5     query = append_to_statement(statement=query, model=SongRelease, filter_=filter_,
    ↪ pagination=pagination, sort=sort)
6
7     songs = (await self.connection.execute(query)).scalars()
8
9     return songs
10
11 @db_error_handler
12 async def create_song(self):
13     created_song = Song()
14     self.connection.add(created_song)
15     await self.connection.commit()
16     await self.connection.refresh(created_song)
17     return created_song

```

SQLAlchemy позволяет сконструировать запрос перед выполнением в базе данных с использованием паттерна ORM. То есть, при конструировании запроса мы напрямую общаемся с полями объекта, а не с рядами таблицы. Затем сам запрос выполняется уже непосредственно при помощи соединения из родительского репозитория. Метод `get_song_releases_short` в листинге 3.13 позволяет получить короткое описание релизов с учётом фильтров, пагинации и сортировки. Метод же `create_song` позволяет создать запись в таблице. Важно отметить, что в отличие от первого метода, он явно вызывает сохранение в базе данных а также обновляет внутреннюю модель при помощи `refresh`.

3.1.2 Валидация входных и выходных параметров, схемы

При рассмотрении листинга 3.13 были упомянуты такие понятия как фильтрация, пагинация и сортировка. Перед рассмотрением сервисов сначала рассмотрим схемы. Схемы определяют, что может прислать пользователь - формат данных, их тип, другие ограничения - а также, что может отдать сам сервер в качестве ответа. Строгое определение схем позволяет документировать поведение интерфейса приложения, а значит воспользоваться механизмами автогенерации кода в других приложениях для обеспечения удобства взаимодействия с сервисом.

Схемы можно классифицировать по потоку данных (входные, выходные данные), а также входные схемы можно классифицировать по их назначению (фильтрация, пагинация, сортировка, доменные данные). Схемы для выходных данных определяют, что сервер должен отдать в качестве ответа. Важно отметить, что поля выходных данных могут (а в идеале, должны) отличаться от названий полей моделей. В качестве примера возьмём схему для получения укороченной информации о песне.

					Разработка алгоритмов и программного обеспечения	Лист
Изм.	Лист	№ докум.	Подп.	Дата		63

Схемы представлены в листинге 3.14

Листинг 3.14 – Схемы для получения укороченной информации о песне

```
1 class AuthorShortOutNested(BaseModel):
2     id: UUID
3     name: str
4
5
6 class GroupShortOutNested(BaseModel):
7     id: UUID
8     name: str
9
10
11 class SongShortOutData(BaseModel):
12     song_id: UUID
13     id: UUID
14     tag: str
15     name: str
16     authors: list[AuthorShortOutNested]
17     groups: list[GroupShortOutNested]
18     duration: int | None
```

Здесь можно увидеть, что схема сильно отличается от того, что находится в базе данных. Если следовать логике базы данных, согласно листингам 3.3 и 3.2, как минимум, в ответе должно было бы гораздо больше данных, а сами данные должны быть гораздо более вложенными. Такое расхождение связано с тем, что схемы создаются под конкретные пользовательские истории. Пользователю не нужно знать вообще всю информацию, ему нужно знать только ту информацию, что представлена в схеме.

Почему просто не отдавать/принимать модели для базы данных? Это в том числе и требования безопасности. Если мы будем использовать одни и те же схемы, существует риск компрометации схемы базы данных, а значит и повышения риска утечки данных. Кроме того, сама модель внутри базы данных может кардинально меняться с течением времени, в то время как разработчики графических интерфейсов будут не готовы отреагировать достаточно быстро на изменения ответов или запросов. Поэтому сервер часто занимается, с первого взгляда, довольно бесполезной задачей переключивания данных из одной схемы в другую. Но такой излишний код в будущем обеспечит расширяемость и поддерживаемость продукта.

На основе этих схем уже создаются оркестрирующие схемы, которые будет непосредственно возвращать сервер. Пример таких ответов находится в листинге 3.15

Листинг 3.15 – Оркестрирующая схема ответа

```

1 class SongListResponse(ApiResponse):
2     message: str = "Song API Response"
3     data: list[SongShortOutData]
4     detail: dict[str, Any] | None = {"key": "val"}
5
6
7 class SongShortResponse(ApiResponse):
8     message: str = "Song API Response"
9     data: SongShortOutData
10    detail: dict[str, Any] | None = {"key": "val"}

```

Входные параметры так же определяются схемами. При создании доменных схем необходимо так же учитывать различные виды входных данных. Некоторые объекты возможно передать только текстовыми данными, вроде общепринятого JSON. Однако некоторые данные можно создать исключительно при помощи FormData API. Это специализированный формат, позволяющий обмениваться файлами между клиентом и сервером. Использование этого API имеет некоторые ограничения. Например, он плохо поддерживает структурированные данные, вроде вложенных объектов и массивов. Для примера, рассмотрим схему создания песни, представленной в листинге [Б.1](#). Эта схема принимает несколько файлов, поэтому `files_file`, `files_leading` и `files_audio_custom_codec` - это массивы. Ввиду ограниченности формата FormData, `files_leading` и `files_audio_custom_codec` приходится вручную разбивать и перепроверять, что каждый из объектов действительно является булевым значением в методах `separate_by_comma_files_leading` и `separate_by_comma_files_audio_custom_codec`. Здесь же можно обнаружить кастомное правило, вроде `check_files_size`, которое позволяет провалидировать равенство размеров массивов. Более простые случаи, описывающие ограничения по длине и типу данных описаны в полях выше, вроде `SongName`.

Ещё один вид входных данных позволяет регулировать результаты выдачи списков домена. Фильтрация, пагинация и сортировка выполняется при помощи расширения пакета Pydantic, с помощью которого и были созданы схемы выше, Pydantic Filters. Этот пакет также предоставляет автоматический инструмент для расширения запросов - в листинге [3.13](#) это вызов метода `append_to_statement`. Сами схемы контроля выдачи данных представлены в листинге [3.16](#)

Листинг 3.16 – Оркестрирующая схема ответа

```

1 class SongFilter(BaseFilter):
2     id__in: list[UUID] | None
3     tag__in: list[str] | None

```

					Разработка алгоритмов и программного обеспечения	Лист
Изм.	Лист	№ докум.	Подп.	Дата		
						65

```

4     bpm__in: list[int] | None
5     key__in: list[str] | None
6     authors_id__in: list[UUID] | None
7     groups_id__in: list[UUID] | None
8     q: str | None = SearchField(target=["name", "tag", "key"])
9
10
11 class SongPagination(PagePagination):
12     pass
13
14
15 SongOrderByLiteral = Literal["id", "name", "tag", "key", "bpm"]
16
17
18 class SongSort(BaseSort):
19     sort_by: SongOrderByLiteral | None = None

```

Так, например, SongFilter умеет сортировать по списку идентификаторов, тегов, BPM и др. А поиск по строке выполняется по полям name, tag и key. Сортировка выполняется по полям id, name, tag, key, bpm.

Таким образом, представленные схемы позволяют контролировать и валидировать поток входящих/выходящих данных.

3.1.3 Сервисы

Основная задача сервисов - это преобразование потока данных. Именно сервисы содержат самую ценную бизнес-логику. Они задают правила, по которым работает приложение, обращаются к хранилищу данных.

Сервисы, как и схемы, создаются из пользовательских сценариев. Например, для песни это могут быть сценарии получения, создания, удаления, изменения. Часто в больших проектах декомпозиция идёт ещё дальше, и каждый отдельный сервис объединяется не по принципу домена, а по принципу набора действий. Так, например, получение песни в текущем приложении можно было бы вынести в отдельный сервис, так как он содержит очень много бизнес-логики, обработку очереди при выборе с учётом тэга. Однако проект не очень большой, поэтому дальнейшую декомпозицию можно будет выполнить при усложнении и дополнении кодовой базы.

Сервис, как и репозиторий, подключается при помощи инъекции, следовательно необходимо предоставить абстрактный сервис а также функцию, которая сможет быть использована в качестве аргумента Depends. Базовый сервис представлен в листинге 3.17. Он содержит опциональное подключение к базе данных для покрытия случаев, когда репозиторий может предоставлять не все необходимые методы.

					Разработка алгоритмов и программного обеспечения	Лист
Изм.	Лист	№ докум.	Подп.	Дата		
						66

Листинг 3.17 – Базовый сервис

```
1 from sqlalchemy.ext.asyncio import AsyncSession
2
3
4 class BaseService:
5     def __init__(self, db: AsyncSession = None):
6         self.db = db
```

Метод для подключения сервиса при помощи инъекции, представленный в листинге 3.19, даже не использует подключение к базе данных. Он возвращает функцию, которая конструирует сервис и которая может быть использована внутри Depends.

Листинг 3.18 – Функция для инъекции сервиса

```
1 from collections.abc import Callable
2
3 from app.services.base import BaseService
4
5
6 def get_service(service_type: type[BaseService]) -> Callable[[], BaseService]:
7     def _get_service() -> BaseService:
8         return service_type()
9
10    return _get_service
```

Методы сервиса можно разделить на два вида: это внутренний метод, а также метод, который непосредственно вызывается извне. Внутренние методы часто оперируют собственными необработанными объектами, в то время методы, вызываемые извне, возвращают схему с ответом.

Одной из функций приложения является предоставления возможности создания продвинутых плейлистов на основе тегов. Рассмотрим в качестве примера функцию, которая выбирает эти треки для плейлиста, представленный в листинге Б.2.

Для начала необходимо получить сам плейлист, для этого есть метод репозитория `get_playlist_by_id`, код которого представлен в листинге ???. Рассмотрим его более детально. Метод `get_playlist_by_id` выбирает плейлисты и песни с указанием `isouter=True` (это значит, что даже если у песни не будет найдено песни, она всё равно будет присоединена). Также присоединяются все релизы песни, относящиеся к песне а также дополнительные данные, включая авторов, группы и файлы. Также необходимо отметить использование `order_by`. Дело в том, что нам необходимо отражать только самые новые релизы, которые соответствуют тэгу, чтобы происходила возможность обновления трека по тэгу. Таким образом происходит автоматическое

					Разработка алгоритмов и программного обеспечения	Лист
Изм.	Лист	№ докум.	Подп.	Дата		67

обновление песни во всех плейлистах. Вызов `filter` позволяет выбирать только необходимые плейлисты, а `limit` обозначает, что будет выбран только один единственный плейлист, что нам и необходимо.

Листинг 3.19 – Метод репозитория для получения плейлиста

```
1 @db_error_handler
2 async def get_playlist_by_id(self, *, playlist_id: UUID) -> Playlist:
3     playlist = (
4         await self.connection.execute(
5             select(Playlist)
6             .join(Playlist.songs, isouter=True)
7             .join(PlaylistSong.song)
8             .join(Song.song_releases, isouter=True)
9             .join(SongRelease.authors, isouter=True)
10            .join(SongRelease.groups, isouter=True)
11            .join(SongRelease.files)
12            .join(SongReleaseFile.file)
13            .order_by(SongRelease.created_at.desc())
14            .filter(Playlist.id == playlist_id)
15            .limit(1)
16        )
17    ).scalar()
18
19    return playlist
```

Вернёмся к листингу [Б.2](#). Основную ценность собой представляет внутренняя функция `track_mapper_function`, задача которого - преобразование треков внутри плейлиста. Как уже было обозначено ранее, плейлист содержит песни со всеми релизами, которые когда-либо выходили. Однако нам необходимо выбрать только одну единственную композицию согласно фильтру. Фильтрация на соответствие выполняется при помощи функции `filter_song_releases_according_to_filter`. Рассмотрим её более подробно в листинге [3.20](#)

Листинг 3.20 – Функция фильтрация релизов по текстовому фильтру

```
1 def filter_song_releases_according_to_filter(songs: list[SongRelease], filter: str) ->
2     ↳ list[SongRelease]:
3     if filter == "":
4         return songs
5
6     return [song for song in songs if str(song.id) == filter or song.tag == filter]
```

Функция устроена очень просто. Если фильтр пустой, то мы возвращаем все песни - всё подходит под пустой фильтр. Иначе же выполняется фильтрация по `id` песни или же по её тегу.

В будущем эта функция может усложняться и дополняться. Несмотря на её

					Разработка алгоритмов и программного обеспечения	Лист
Изм.	Лист	№ докум.	Подп.	Дата		68

крайне маленький размер, она представляет критическую ценность для проекта.

Задача фильтрации композиций в плейлисте имеет большое значение для проекта. В будущем при увеличении количества пользователей необходимо предусмотреть функции кеширования ответов при помощи установления заголовков Cache-Control или же при помощи отдельного сервиса, который предоставляет возможность хранения данных прямо в оперативной памяти - Redis. Необходимо также рассмотреть возможность введения различных способов индексации и переноса этой логики в слой репозитория, так как линейный поиск может быть достаточно медленным.

Вернёмся к листингу Б.2. Если полученный список треков пустой, он возвращает модель описывающую отсутствие трека. Нам всё ещё важно показывать даже отсутствующие треки на случай, если они не найдены, чтобы пользователь мог понять, что он, например, опечатался при введении фильтра. Если же список непустой, функция берёт самый свежий трек и получает ведущий файл у этого трека. После получения всей необходимой информации внутренняя функция выполняет преобразование и возвращает схему.

Рассмотрим листинг 3.21, в котором представлен метод `get_leading_audio_file`.

Листинг 3.21 – Метод для получения ведущего аудиофайла

```
1 def get_leading_audio_file(self, files: list[SongReleaseFile]) -> SongReleaseFile | None:
    ↪
2     return next((x for x in files if x.primary and x.file.mime.startswith("audio/")),
    ↪ None)
```

Это тоже относительно простой метод. Вычисление ведущего файла выполняется при помощи значения поля `primary` а также его `mime`-типа.

Существует также необходимость определения главного изображения, что позволит задать изображение для релиза композиции. В этом методе логика аналогичная, однако проверка `mime`-типа выполняется уже с другим значением `"image/"`.

Таким образом, сервис преобразует данные и инкапсулирует внутри себя всю бизнес-логику.

3.1.4 Внешний интерфейс

Внешний интерфейс может быть представлен различными способами. Это может быть и графическое приложение, и программа для работы с командной строкой. Раннее описанная декомпозиция позволяет подключить любой интерфейс для взаимодействия с сервисами. Согласно архитектуре приложения, интерфейсом для взаимодействия был выбран HTTP, следовательно для взаимодействия с пользователем используются HTTP эндпоинты. FastAPI предоставляет быстрый и лёгкий

					Разработка алгоритмов и программного обеспечения	Лист
Изм.	Лист	№ докум.	Подп.	Дата		
						69

способ для создания таких эндпоинтов при помощи роутеров.

Роутер самого высокого уровня определяется в файле `__init__.py`, содержимое этого файла представлено в листинге 3.22, который уже включает в себя роутеры из дочерних файлов.

Листинг 3.22 – Верхнеуровневый роутер

```
1 from fastapi import APIRouter
2
3 from app.api.v1 import auth, author, files, group, playlist, song, users
4
5 api_router = APIRouter()
6 api_router.include_router(auth.router, prefix="/auth", tags=["auth"])
7 api_router.include_router(users.router, prefix="/users", tags=["users"])
8 api_router.include_router(files.router, prefix="/files", tags=["files"])
9 api_router.include_router(author.router, prefix="/authors", tags=["authors"])
10 api_router.include_router(group.router, prefix="/groups", tags=["groups"])
11 api_router.include_router(song.router, prefix="/songs", tags=["songs"])
12 api_router.include_router(playlist.router, prefix="/playlists", tags=["playlists"])
```

Одна из особенностей при объявлении роутера - это версионирование. Так, в текущем приложении представлена только API версии 1. Однако в будущем после большого рефакторинга схем может потребоваться введение совершенно другой версии API, которая сильно будет отличаться от предыдущей версии. Но для сохранения возможности взаимодействия с сервисом и постепенной миграции фронтенда может оставаться относительно неизменное API предыдущей версии.

Сами эндпоинты объявляются при помощи декоратора, который регистрирует этот эндпоинт. Эндпоинт представляет собой обычную функцию, которая возвращает объект соответствующего типа. Пример такого эндпоинта описан в листинге 3.23

Листинг 3.23 – Пример эндпоинта

```
1 router = APIRouter()
2
3 @router.get("/{author_id}", status_code=HTTP_200_OK, responses=ERROR_RESPONSES, name="
  ↳ get_author", response_model=AuthorResponse)
4 async def get_author(
5     *, author_service: AuthorsService = Depends(get_service(AuthorsService)),
6     ↳ author_repository: AuthorsRepository = Depends(get_repository(AuthorsRepository)),
7     ↳ author_id: UUID
8 ) -> AuthorResponse:
9     response = await author_service.get_author_by_id(author_id=author_id, author_repo=
  ↳ author_repository)
10
11     return await handle_result(response)
```


В эндпоинте задаётся тип возвращаемых данных, коды ошибок, в него внедряются все необходимые зависимости. Содержимое сервиса часто тривиально и представляет собой вызов метода внутри сервиса.

FastAPI предоставляет возможность автоматической генерации документации на основании описанных роутеров для HTTP. Он генерирует OpenAPI документацию, представляющую из себя JSON файл с описанием API, а затем инструменты вроде Swagger-UI позволяют сгенерировать сайт, позволяющий просмотреть содержимое API и взаимодействовать с ним. Фрагмент документации представлен на рисунке 3.1.

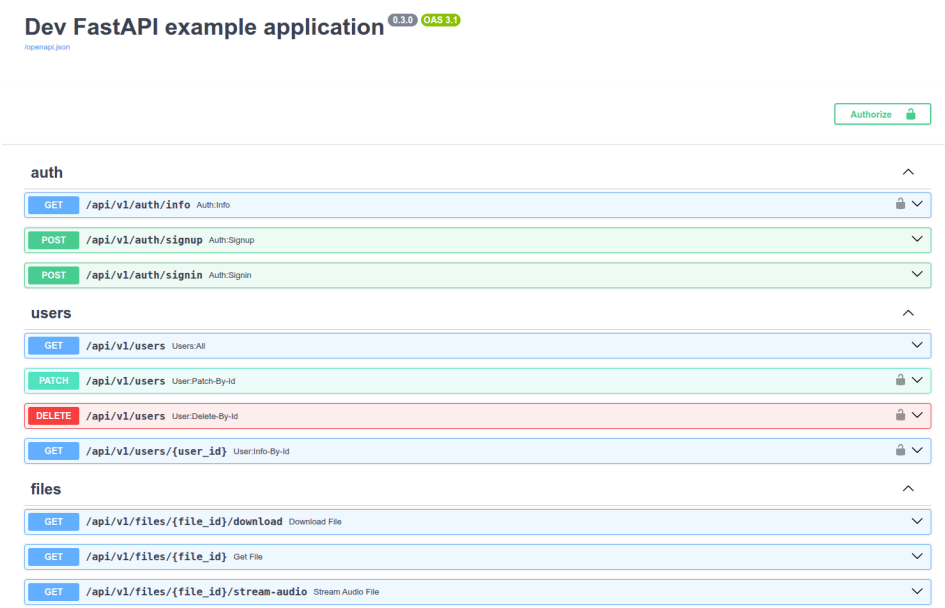


Рисунок 3.1 – Документация API, авторский материал

3.1.5 Авторизация

Существует несколько основных способов реализации авторизации в веб-приложениях. В данном приложении для авторизации пользователя используется заголовок Header со сгенерированным токеном JWT, его структура представлена на рисунке 3.2.

JWT токен - это строка, которая состоит из трёх компонентов. Первый компонент - это описание самого JWT токена. В нём может храниться способ шифрования. Второй компонент - это то, что позволяет идентифицировать пользователя. Обычно он содержит идентификационные данные, вроде уникального ID, почты, имени и другой информации о пользователе. Третий компонент - это подпись. Подпись, используя алгоритм из первого сегмента, преобразует соединённый заголовок и описание токена через точку с использованием некоторого секретного ключа. Шифрование может быть как симметричным, так и ассиметричным. Алгоритмы с симметричным шифрованием, вроде HS256, для кодирования и декодирования используют

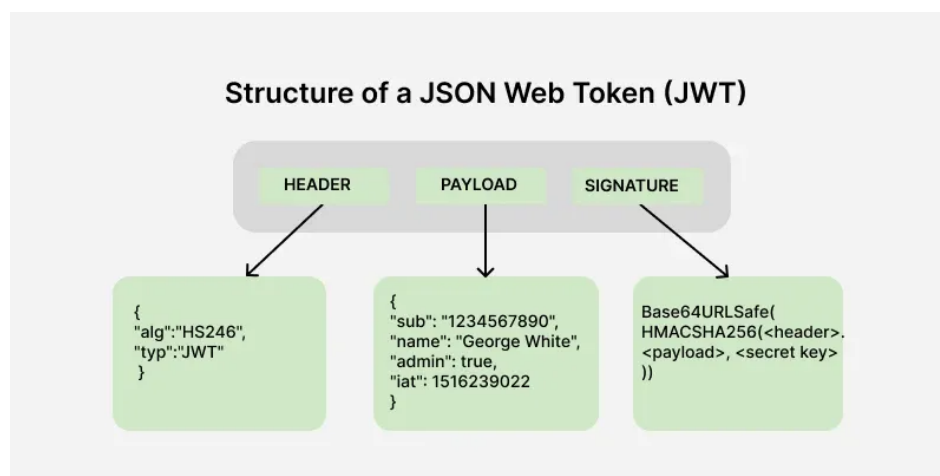


Рисунок 3.2 – Структура JWT-токена [6]

один и тот же секретный ключ. При этом способе у конечного пользователя нет никакого способа проверить подпись, а сам секретный ключ должен храниться исключительно в приложении, которое занимается генерацией и проверкой токена. Безопасность и удобство таких методов ниже, так как валидацию ключа может выполнить только сервер. Однако такой способ математически проще.

При асимметричном шифровании приватный ключ позволяет зашифровать информацию, а публичный - расшифровать. Приватный ключ хранится на сервисе, который позволяет авторизоваться, а компрометация приватного ключа несёт риск, так как злоумышленник сможет генерировать ключи в любой момент. Публичный ключ же доступен любому. Он позволяет декодировать подпись и, сравнив её с содержимым первых двух сегментов JWT токена, проверить подлинность подписи. Такие алгоритмы математически являются более сложными.

В приложении в качестве алгоритма при генерации подписи выбран симметричный HS256. Так как приложение не является микросервисным и выполняет свою конкретную задачу, в нём можно консолидировать задачи выдачи токенов и проверки, а получение информации о пользователе можно выполнять через специальный эндпоинт. Большой необходимости в асимметричном шифровании нет.

Библиотека FastAPI не предоставляет собственного инструмента для внедрения аутентификации, это минималистичная библиотека, которая выполняет функции именно создания API. Для возможности идентификации пользователя необходима разработка собственного сервиса.

Листинг 3.24 – Функции для создания JWT

```
1 def create_token(
2     *,
3     content: dict[str, str],
```

```

4     secret_key: str,
5     expires_delta: timedelta | None = timedelta(minutes=36000),
6 ):
7     to_encode = content.copy()
8     expire = datetime.now(UTC) + expires_delta
9     to_encode.update(TokenBase(exp=expire, sub=JWT_SUBJECT).model_dump())
10
11     encoded_jwt = jwt.encode(to_encode, secret_key, algorithm=ALGORITHM)
12     return encoded_jwt
13
14 def create_token_for_user(user: User, secret_key: str) -> UserTokenData:
15     token_user_dict = TokenUser(id=user.id, username=user.username, email=user.email).
16     ↪ model_dump()
17     created_token = create_token(
18         content=token_user_dict,
19         secret_key=secret_key,
20     )
21     return UserTokenData(access_token=created_token, token_type=TOKEN_TYPE)

```

В листинге 3.24 описаны методы, которые позволяют создать JWT токен, позволяющий идентифицировать пользователя. Для создания токена используется библиотека Python JWT. Полезное содержимое токена включает идентификатор, имя пользователя а также его почту. Эти методы могут использоваться в пользовательских сценариях регистрации и авторизации. В листинге 3.25 представлен обратный процесс декодирования JWT токена с помощью той же библиотеки JWT с указанием секретного ключа и алгоритма.

Листинг 3.25 – Функции для декодирования JWT

```

1 def get_user_from_token(token: str, secret_key: str) -> str:
2     try:
3         decoded_user = jwt.decode(token, secret_key, algorithms=ALGORITHM)
4         return TokenUser(**decoded_user)
5
6     except JWTError as decode_error:
7         raise ValueError("unable to decode") from decode_error
8     except ValidationError as validation_error:
9         raise ValueError("invalid token") from validation_error

```

Необходимо также рассмотреть вопрос хранения пользователя внутри базы данных и проверки его пароля. Пароль пользователя хранится в виде хэша и соли - специальной строки, котоая пе

ПРИЛОЖЕНИЕ А

Техническое задание

А.1 Введение

Наименование: BackTrack

Краткая характеристика области применения: управление процессами разработки музыкальных композиций

А.2 Основания для разработки

Выполнение бакалаврской дипломной работы при обучении в БГТУ им. В. Г. Шухова

А.3 Назначение разработки

Хранение, версионирование и распространение аудиоданных

А.4 Требования к программе или программному изделию

А.4.1 Требования к функциональным характеристикам

- а) Должна производиться аутентификация пользователя при использовании сервисом
- б) Авторы
 - 1) Автор должен содержать имя, аватар, описание
 - 2) Должны поддерживаться операции добавления, удаления, редактирования, получения списка авторов и получение конкретного автора с его данными
- в) Группы
 - 1) Группа должна включать в себя аватар, описание и авторов внутри этой группы
 - 2) Должны поддерживаться операции добавления, удаления, редактирования и получения списка групп а также получения информации о конкретной группе
- г) Песни
 - 1) Песня содержит в себе имя, описание, тэг, ВРМ, тональность, слова, авторов и группы а также файлы этой песни.

					ПРИЛОЖЕНИЕ А			
Изм.	Лист	№ докум.	Подп.	Дата	Разработка программного обеспечения для прогнозирования погоды на основе анализа спутниковых снимков с использованием методов машинного обучения	Лит.	Лист	Листов
Разраб.		Пахомов В.А.		3.06.26				
Руковод.		Осипов О.В.		3.06.26			74	80
Консул.						БГТУ им. В.Г. Шухова, ПБ-223		
Н. контр.		Бондаренко Т.В.		11.06.26				
Зав. каф.		Поляков В.М.		25.06.26				

- 2) Должна быть возможность указать ведущие файлов. Эти файлы репрезентируют песню. Так, ведущее изображение является картинкой этой песни. А ведущее аудио - проигрывающей песней при её запуске. Дополнительные файлы могут содержать прочую важную информацию.
- 3) После создания песню удалить нельзя - можно только выпустить следующий релиз песни. Всегда можно просмотреть предыдущие релизы песен.
- 4) Должен быть доступен чат внутри каждой из песен, где пользователи могут просматривать а также отправлять свои собственные сообщения
- 5) Должна быть реализована возможность просмотра конкретной версии песни.

д) Плейлист

- 1) Плейлист собирает в себе песни и фильтрует их по фильтру. Плейлист должен в себя включать изображение, название и описание.
- 2) Функция фильтрации заключается в поиске по значению, которое может представлять собой либо название тега либо название версии. Если указано название тега, в песне должна быть найдена наиболее новая песня с указанным тегом. Если фильтр не указан, будет предоставлена последняя версия песни. Если же указан номер версии, будет найдена версия с этим конкретным номером.
- 3) Должны быть реализованы операции добавления, удаления, редактирования и получения

е) Взаимодействие с файлами

- 1) Должен быть предоставлен альтернативный способ хранения аудиоданных, использующий внутренний кодек для хранения аудиоданных.
- 2) Необходимо предоставить возможность скачивать загруженные файлы
- 3) Отправка аудиоданных со своим кодеком должна поддерживать специальные заголовки, позволяющие получать данные порционно. Распаковка не должна происходить одномоментно.

ж) Проигрыватель

- 1) Должен быть реализован проигрыватель, позволяющий проиграть плейлист или же отдельную версию песни.
- 2) Плейлист должен поддерживать следующие операции: выбрать порядок в пределах плейлиста (перемешать, в обычном порядке, зациклить), выбрать порядок в пределах песни (зациклить песню или разрешить переход к следующим), выбрать следующий трек, выбрать предыдущий трек, отразить

информацию о песне, выбрать громкость.

А.4.2 Требования к надёжности

- а) Валидация входных и выходных данных
 - 1) Данные на входе и выходе должны валидироваться в соответствии с требованиями к значениям сущности, а также экранироваться
- б) Проверка работоспособности сервисов
 - 1) Сервер должен определить эндпоинт, позволяющий проверить его работоспособность (healthcheck)
 - 2) Сервер, отдающий статические файлы должен определить эндпоинт, позволяющий проверить его работоспособность (healthcheck)
 - 3) Необходимо периодически проверять работоспособность сервисов
- в) Время восстановления после отказа после обнаружения неисправности не должно превышать 1 час

А.4.3 Условия эксплуатации

Особых условий эксплуатации не требуется

А.4.4 Требования к составу и параметрам технических средств

- а) Скорость канала клиента: 10 МБит/с
- б) Скорость канала сервера: 10 ГБит/с
- в) Наличие на сервере CUDA совместимого ускорителя с минимумом 16 Гб видеопамяти
- г) Минимум 32 Гб оперативной памяти на сервере
- д) 1 терабайт места на сервере

А.4.5 Требования к информационной и программной совместимости

- а) Версия поддерживаемой CUDA: как минимум 13.0

А.4.6 Требования к маркировке и упаковке

Особых требований не предоставлено

А.5 Требования к программной документации

Состав документации должен описывать описание работы с сервисом через графический интерфейс

А.6 Техничко-экономические показатели

Аренда минимального сервера с графическим ускорителем составляет 27300 руб/мес. Альтернатив доступных на российском рынке со схожим функционалом не представлено. Подписка на зарубежный сервис Voombox.io для команды из сотни человек составляет 33600 руб/мес.

Годовая потребность поддержки сервера составит 327600 рублей.

А.7 Стадии и этапы разработки

- а) Планирование
- б) Проектирование
- в) Разработка
- г) Тестирование
- д) Внедрение и сопровождение

А.8 Порядок контроля и приёмки

В качестве тестирования продукта используется Smoke-тестирование а также оценка шумности аудиокодека.

					ПРИЛОЖЕНИЕ А	Лист
Изм.	Лист	№ докум.	Подп.	Дата		77

ПРИЛОЖЕНИЕ Б

Листинги разработанных схем для создания композиции

Листинг Б.1 – Схема создания композиции

```

1 SongName = Field(min_length=1, max_length=1024)
2 SongTag = Field(min_length=0, max_length=256)
3 SongDescription = Field(min_length=0, max_length=8192)
4 SongKey = Field(min_length=1, max_length=32)
5 SongLyrics = Field(min_length=0, max_length=8192)
6 SongBPM = Field(gt=0)
7
8
9 # Модели для создания
10 class SongInCreate(SongBase):
11     name: str = SongName
12     tag: str | None = SongTag
13     description: str = SongDescription
14     bpm: int | None = SongBPM
15     key: str | None = SongKey
16     lyrics: str | None = SongLyrics
17     files_file: list[UploadFile]
18     files_leading: list[bool]
19     # Использовать кастомный кодек для хранения аудио
20     files_audio_custom_codec: list[bool]
21     authors: list[UUID]
22     groups: list[UUID]
23
24     @field_validator("files_leading", mode="before")
25     def separate_by_comma_files_leading(cls, value):
26         if isinstance(value, list) and isinstance(value[0], str):
27             return [bool(leading) for leading in value[0].split(",")]
28
29         return value
30
31     @field_validator("files_audio_custom_codec", mode="before")
32     def separate_by_comma_files_audio_custom_codec(cls, value):
33         if isinstance(value, list) and isinstance(value[0], str):
34             return [bool(leading) for leading in value[0].split(",")]
35
36         return value
37
38     @model_validator(mode="after")
39     def check_files_size(self) -> "SongInCreate":
40         if len(self.files_file) != len(self.files_leading) or len(self.files_file) !=

```

					ПРИЛОЖЕНИЕ Б			
Изм.	Лист	№ докум.	Подп.	Дата	Разработка программного обеспечения для прогнозирования погоды на основе анализа спутниковых снимков с использованием методов машинного обучения	Лит.	Лист	Листов
Разраб.	Пахомов В.А.			3.06.26				
Руковод.	Осипов О.В.			3.06.26			78	80
Консул.						БГТУ им. В.Г. Шухова, ПБ-223		
Н. контр.	Бондаренко Т.В.			11.06.26				
Зав. каф.	Поляков В.М.			25.06.26				

```

41     ↪ len(self.files_audio_custom_codec):
42         raise ValueError("Files array and files leadings should be of a same size")
43     return self

```

Листинг Б.2 – Метод получения содержимого плейлиста

```

1  @return_service
2  async def get_playlist_by_id(
3      self, playlist_id: UUID, playlist_repo: PlaylistsRepository = Depends(get_repository(
4          ↪ PlaylistsRepository)), song_service: SongsService = Depends(get_service(SongsService
5          ↪ ))
6  ) -> PlaylistDetailedResponse:
7      playlist = await playlist_repo.get_playlist_by_id(playlist_id=playlist_id)
8      if not playlist:
9          return response_4xx(
10             status_code=HTTP_404_NOT_FOUND,
11             context={"reason": constant.FAIL_PLAYLIST_NOT_FOUND},
12         )
13
14     def track_mapper_function(track: PlaylistSong):
15         fitting_release = filter_song_releases_according_to_filter(track.song.song_releases,
16             ↪ track.filter)
17         if len(fitting_release) == 0:
18             return PlaylistExtendedOutTracks(
19                 id=track.song.id,
20                 version=None,
21                 name="Песня не найдена. Уточните фильтр",
22                 filter=track.filter,
23                 groups=[],
24                 authors=[],
25                 duration=None,
26                 sound_file_id=None,
27             )
28
29         fitting_track = fitting_release[-1]
30
31         leading_audio = song_service.get_leading_audio_file(fitting_track.files)
32
33         return PlaylistExtendedOutTracks(
34             id=track.song.id,
35             version=fitting_track.id,
36             name=fitting_track.name,
37             filter=track.filter,
38             groups=[PlaylistExtendedOutGroups(id=group.id, name=group.name) for group in
39                 ↪ fitting_track.groups],
40             authors=[PlaylistExtendedOutAuthors(id=author.id, name=author.name) for author in
41                 ↪ fitting_track.authors],
42             duration=leading_audio.file.duration if leading_audio else None,
43             sound_file_id=leading_audio.file_id if leading_audio else None,
44         )

```



```

40
41 return dict(
42     status_code=HTTP_200_OK,
43     content={
44         "message": constant.SUCCESS_PLAYLIST_FOUND,
45         "data": jsonable_encoder(
46             PlaylistExtendedOutData(
47                 id=playlist.id, name=playlist.name, description=playlist.description, file_id=
↪ playlist.file_id, tracks=[track_mapper_function(track) for track in playlist.songs]
48             )
49         ),
50     },
51 )

```

					ПРИЛОЖЕНИЕ Б	Лист
Изм.	Лист	№ докум.	Подп.	Дата		80

СЛУЖЕБНАЯ ИНФОРМАЦИЯ

Список иллюстраций

1.1	Пример структуры автоэнкодера [1]	19
1.2	Удаление шума при помощи автоэнкодера [2]	20
1.3	Принцип работы Convolutional Layer [3]	21
1.4	Запись голоса человека в спектрограмме, авторский материал	23
2.1	Схема создания и использования аутентификационного токена [4]	36
2.2	Схема автоэнкодера, авторский материал	38
2.3	Заголовок файла, авторский материал	39
2.4	Заголовок сэмпла, авторский материал	40
2.5	Структура базы данных, авторский материал	41
2.6	Много микросервисов [5]	44
2.7	Архитектура сервера, авторский материал	45
2.8	Архитектура модуля подготовки аудиоданных, авторский материал	46
2.9	Архитектура модуля обратного преобразования аудиоданных, авторский материал	47
2.10	Архитектура модуля обучения, авторский материал	48
2.11	Взаимодействие между модулями внутри Audio Manager, авторский материал	49
2.12	Модуль audio_manager, авторский материал	50
2.13	Архитектура фронтенд приложения, авторский материал	54
3.1	Документация API, авторский материал	71
3.2	Структура JWT-токена [6]	72

Список картинок нужен только для прохождения нормоконтроля.

В диплом он не вшивается!

					СЛУЖЕБНАЯ ИНФОРМАЦИЯ			
Изм.	Лист	№ докум.	Подп.	Дата				
Разраб.	Пахомов В.А.			3.06.26	Разработка программного обеспечения для прогнозирования погоды на основе анализа спутниковых снимков с использованием методов машинного обучения	Лит.	Лист	Листов
Руковод.	Осипов О.В.			3.06.26			1	80
Консул.						БГТУ им. В.Г. Шухова, ПБ-223		
Н. контр.	Бондаренко Т.В.			11.06.26				
Зав. каф.	Поляков В.М.			25.06.26				

СЛУЖЕБНАЯ ИНФОРМАЦИЯ

Список таблиц

1.1	Сравнительная таблица сервисов хранения аудиоданных	13
1.2	Сравнительная таблица кодеков	16

Список таблиц нужен только для прохождения нормоконтроля.

В диплом он не вшивается!

					СЛУЖЕБНАЯ ИНФОРМАЦИЯ			
Изм.	Лист	№ докум.	Подп.	Дата				
Разраб.	Пахомов В.А.			3.06.26	Разработка программного обеспечения для прогнозирования погоды на основе анализа спутниковых снимков с использованием методов машинного обучения	Лит.	Лист	Листов
Руковод.	Осипов О.В.			3.06.26			2	80
Консул.						БГТУ им. В.Г. Шухова, ПБ-223		
Н. контр.	Бондаренко Т.В.			11.06.26				
Зав. каф.	Поляков В.М.			25.06.26				

СЛУЖЕБНАЯ ИНФОРМАЦИЯ

Листинги

3.1	Базовый класс для моделей базы данных	56
3.2	Базовый класс для моделей базы данных	56
3.3	Модель релиза песни	57
3.4	Простая модель для обеспечения связи N к N	57
3.5	Усложнённая модель для обеспечения связи N к N	58
3.6	Экспорт моделей	59
3.7	Конфигурация Alembic	59
3.8	Конфигурация окружения Alembic	60
3.9	Пример миграции Alembic	60
3.10	Инициализация соединения с базой данных	61
3.11	Функции для инъекции зависимостей для работы с базой данных .	61
3.12	Базовый репозиторий	62
3.13	Методы в репозитории песен	62
3.14	Схемы для получения укороченной информации о песне	64
3.15	Оркестрирующая схема ответа	65
3.16	Оркестрирующая схема ответа	65
3.17	Базовый сервис	67
3.18	Функция для инъекции сервиса	67
3.19	Метод репозитория для получения плейлиста	68
3.20	Функция фильтрация релизов по текстовому фильтру	68
3.21	Метод для получения ведущего аудиофайла	69
3.22	Верхнеуровневый роутер	70
3.23	Пример эндпоинта	70
3.24	Функции для создания JWT	72
3.25	Функции для декодирования JWT	73
Б.1	Схема создания композиции	78
Б.2	Метод получения содержимого плейлиста	79

Список листингов нужен только для прохождения нормоконтроля.

В диплом он не вшивается!

					СЛУЖЕБНАЯ ИНФОРМАЦИЯ			
Изм.	Лист	№ докум.	Подп.	Дата				
Разраб.	Пахомов В.А.			3.06.26	Разработка программного обеспечения для прогнозирования погоды на основе анализа спутниковых снимков с использованием методов машинного обучения	Лит.	Лист	Листов
Руковод.	Осипов О.В.			3.06.26			80	80
Консул.						БГТУ им. В.Г. Шухова, ПБ-223		
Н. контр.	Бондаренко Т.В.			11.06.26				
Зав. каф.	Поляков В.М.			25.06.26				